

A software defect prediction method using binary gray wolf optimizer and machine learning algorithms

Hao Wang^a, Bahman Arasteh^{b,c,*}, Keyvan Arasteh^b,
Farhad Soleimani Gharehchopogh^d, Alireza Rouhi^e

^a School of Computer and Information Technology, Beijing Jiaotong University, Beijing, China

^b Department of Software Engineering, Faculty of Engineering and Natural Science, Istinye University, Istanbul, Türkiye

^c Department of Computer Science, Khazar University, Baku, Azerbaijan

^d Department of Computer Engineering, Urmia Branch, Islamic Azad University, Urmia, Iran

^e Faculty of Information Technology and Computer Engineering, Azarbaijan Shahid Madani University, Tabriz, Iran

ARTICLE INFO

Keywords:

Software defect prediction
Feature selection
Binary gray wolf optimizer
Machine learning
Module classification

ABSTRACT

Context: Software defect prediction means finding defect-prone modules before the testing process which will reduce testing cost and time. Machine learning methods can provide valuable models for developers to classify software faulty modules.

Problem: The inherent problem of the classification is the large volume of the training dataset's features, which reduces the accuracy and precision of the classification results. The selection of the effective features of the training dataset for classification is an NP-hard problem that can be solved using heuristic algorithms.

Method: In this study, a binary version of the Gray Wolf optimizer (bGWO) was developed to select the most effective features of the training dataset. By selecting the most influential features in the classification, the precision and accuracy of the software module classifiers can be increased.

Contribution: Developing a binary version of the gray wolf optimization algorithm to optimally select the effective features and creating an effective defect predictor are the main contributions of this study. To evaluate the effectiveness of the proposed method, five real-world and standard datasets have been used for the training and testing stages of the classifier.

Results: The results indicate that among the 21 features of the train datasets, the basic complexity, sum of operators and operands, lines of codes, number of lines containing code and comments, and sum of operands have the greatest effect in predicting software defects. In this research, by combining the bGWO method and machine learning algorithms, accuracy, precision, recall, and F1 criteria have been considerably increased.

1. Introduction

The presence of a fault in the software system is considered a serious threat to software quality. Software defect prediction is one of the important fields in software engineering, which is used to improve software quality by finding software defects [1]. To increase the quality of the software, defects should be detected before its release. Considering the applicability of the Pareto principle, most

* Corresponding author.

E-mail address: Bahman.arasteh@istinye.edu.tr (B. Arasteh).

software defects occur in small parts of modules, the prediction and detection of defects in the early stages of software development will increase the quality of the produced software artifact. Software defect prediction means finding defect-prone modules before the testing process which will reduce testing cost and time. Since the size and complexity of the software systems are increasing and the exhaustive test of all the software modules is practically impossible, predicting the prone to defect modules increases the software quality. Machine learning methods can provide valuable models for developers to classify software faulty modules [2].

To increase software quality, the use of software defect prediction methods has increased recently. In this study, software defect prediction focuses on the classification of software components (modules) into two classes, prone to defects and non-prone to defects [1,2]. In the classification method, according to the history of the defective module, a model is extracted that will be used to predict the defects more accurately in the new modules. Previous findings based on learning from metrics obtained from software modules show a strong relationship between these metrics and defect prediction [3]. There are different algorithms for software module classification. Among these methods, we can mention the **decision tree**, **k-nearest neighbor algorithm**, **Naive Bayes algorithm**, **support vector machine**, and **neural networks** to name a few. The inherent problem of the classification is the large volume of features, which reduces the accuracy of the classification results. To select and resolve the mentioned problem and reduce the dimensions of the features, **feature selection methods** are used. The selection of effective features for classification is an NP-hard problem that can be solved using **evolutionary algorithms** [4].

In this paper, a new method was proposed for predicting software defects; in the first stage, a **binary version of the Gray Wolf optimizer (bGWO)** was developed to select the most effective features of the training dataset. Then, different machine learning algorithms have been used to train a classification model using the created optimal training set. The bGWO is based on swarm intelligence. This optimization algorithm has been designed to inspire the behavior of wolves in nature by finding their prey in an artificial system. This paper aims to use the ability of bGWO to select the most important features to predict software defects using machine learning algorithms. Then, machine learning algorithms are used to uncover and rectify software defects before the software has been released. Different machine learning algorithms such as K-nearest neighbors (KNN), decision tree (DT), support vector machine (SVM), Naive Bayes (NB), and artificial neural networks (ANN) have been used to create faulty module classification. By selecting the most influential features in the classification, the precision and accuracy of the software module classifier can be increased. At the final stage, the testing stage, the performance of the suggested feature selection method was evaluated by the test data. In this research, the goals are as follows:

- Determining the most effective features of software fault prediction datasets
- Increasing the software fault predictors' accuracy, precision, and sensitivity
- Increasing the software fault predictors' performance and stability

The main contributions of this study are:

- Developing a specific binary version of the gray wolf optimization algorithm to select the most effective features of the fault prediction dataset.
- Creating an effective and efficient classification model to detect faulty modules of the software.
- Increasing the efficiency of the software fault detection methods by selecting the smallest subset and the most effective features.

In the following, **Section 2** reviews the related works on the problem of software defects prediction. In **Section 3**, the details of the proposed method are presented. **Section 3** includes two subsections; in the first subsection, the binary version of the GWO algorithm was developed and used to select the most effective features in the training dataset. In the second subsection of **Section 3**, the desired classifier was created using the optimal train set and different machine learning algorithms. **Section 4** presents all the relevant results from the tests conducted with the specified criteria on the real-world datasets. Finally, **Section 5** concludes the paper and gives future work guidelines.

2. Related works

In the software development process, the testing phase is very important [5-7]. The reliability of a software is a function of resident bugs¹ (faults) in the software [8,9]. This process is very expensive in terms of budget and time, so predicting the defect-prone software modules is critical. The prediction is performed before the beginning of the test phase. As a result, the prone-to-defect modules are identified and corrected. Based on the recorded history of the problematic software modules in the previous implementation of the project or similar projects, a model is extracted that can help predict the appropriate accuracy of the defects in the newly created modules. The research conducted in the field of defect prediction and estimation shows that the hypothesis used in creating the model has a great impact on the efficiency of the prediction model [10]. There are different approaches for software fault prediction. In this section, **four machine learning methods, including normalization-based methods, unbalanced learning-based methods, feature selection-based methods, and blended learning-based methods, are reviewed.**

¹ In this paper, the terms "fault", "bug" and "mistake" have been used in the same meaning and can be used interchangeably.

2.1. Normalization learning methods

Data normalization is a pre-processing that is performed on datasets before training and testing. Data normalization is used to standardize the values of independent features of the dataset. Since the ranges of different feature values in datasets are different, the objective function of some machine learning algorithms does not work well without normalization. Normalization has a positive effect on the accuracy of algorithms. Here, the measured values are mapped to a new range [11]. In most of the research conducted in the field of software defect estimation, three methods of **logarithmic normalization**, **min-max**, and **Z-standardization** have been used. In the logarithmic normalization method, all sample feature values are mapped to their logarithms. In the min-max method, the minimum and maximum values of each sample feature are found, and each sample feature is normalized between these two maximum and minimum values. In the Z-standard method, the mean and standard deviation of feature values are used to map feature values to a new range with a mean of 0 and a standard deviation of 1. For example, the logarithmic normalization, min-max, and Z-standard methods have been used in [12], respectively. In classification problems, the number of samples in each class can be different.

In binary classification problems, the imbalance problem occurs when one class has many samples, while the other class is represented by a small number of samples. Because most learning algorithms train the classifier based on an equal number of training samples in all classes, when the algorithms are applied to unbalanced datasets, the classifier will have a very poor prediction and estimate of the minor class. Therefore, for problems with unbalanced classes, several techniques have been proposed which are classified into three categories: **data-level approach**, **algorithm-level approach**, and **cost-sensitive learning approach** [13]. Data-level approaches by resampling from the data space with no side-effects on the learning algorithm change the data distribution so that the imbalance effects are removed in the pre-processing stage, i.e., sampling will be the pre-processing step to balance the distribution of classes in unbalanced datasets. The training samples are modified by different methods to make a balanced distribution of the data and allow the classifiers to run on their standard form [13]. Data sampling methods are divided into two main categories: increasing sampling rate and decreasing sampling rate. The first category tries to balance the data by picking more samples from the minor class, while the second category balances the data set by removing some samples from the major class. On the other hand, algorithm-level approaches help learning algorithms direct the learning process toward the minor class. Bagging and boosting learning algorithms can be included in this category.

2.2. Feature selection-based methods

Machine learning methods that deal with high-dimensional data, like a large number of features in the dataset, face many problems. Some of these problems are time and space complexity, the difficulty of extracting knowledge, and the need to control complexity to avoid overfitting in training [14]. Therefore, dimensionality reduction methods represent datasets in a space with a lesser dimension in such a way that the main properties of the dataset in the original space are preserved. Dataset dimension reduction is divided into two categories: methods based on feature extraction and methods based on selecting a subset of features. These methods map the multidimensional space of the data to a space with fewer dimensions. In other words, by combining the values of the existing features, they create a smaller number of features such that these features have the most information about the primary features. One of the most popular feature extraction methods is the Principal Component Analysis (PCA).

Selecting a subset of features is one of the most widely used methods in the field of machine learning. This method is very important in many applications such as classification and regression. In these applications, normally there exist many features, many of which are either ineffective or reduce the accuracy of learning. Eliminating these features, in addition to increasing accuracy, reduces computational complexity. In feature selection, the goal is to find a subset with the minimum number of features appropriate for the intended application. Feature selection methods include filtering and classification methods in the defect prediction field [15]. In the filtering method, no classification function is used. In other words, no feedback from the applied learning algorithm will be used. This method is applied independently of the machine learning algorithm. Feature subsets are evaluated by other concepts. In the first step, a credit rating is calculated for each feature. These ranks are then sorted, and features with the lowest ranks are removed (a threshold is used for feature ranks). The resulting subset of features feeds into the classification system. Fig. 1 shows the procedure for filtering methods.

Filter-based feature ranking techniques called information gain (IG), information gain rate (GR), symmetric uncertainty (SU), chi-square test (CS), and two types of relief have been used in [16]. Filter CS aims to test the distribution of classes and evaluate the correlation of features. An IG filter is information about a specific feature (feature Y) obtained by the target class and the value of another feature (feature X). The weakness of IG is that it prefers features with a large number of values, i.e., the probability of using feature information with higher values is higher than the feature with lower values, even if the feature with lower values has more useful information. The GR and SU methods resolve the weakness of IG. The GR method penalizes multivalued features, and the SU method does this by dividing the sum of the entropies of X and Y. Relieff filter is also an extension of Relief that can manage noise and multiclass datasets.

The set of methods that use the evaluation function based on the error rate of the learning method are called classification or black-

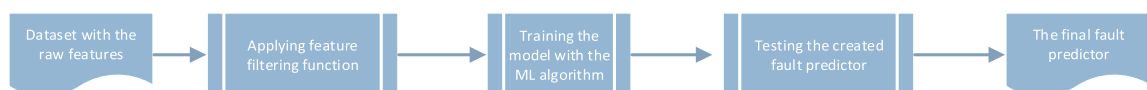


Fig. 1. The procedure of the filter-based method for feature selection.

box methods. In this method, each new subset of features is created by the generator function and trained by the machine learning method. The number of errors in the test set, or the error rate of the learning method, determines the score of the subset. In general, the classification method has a better performance than the filter method, but it is computationally complex compared to the filter method. To find the optimal set of features, there are two selection methods: forward selection and backward selection. Forward selection examines the features that are selected at the same time for entry at each step. The backward selection method includes all features of the model and performs elimination until the stopping condition is satisfied. In this way, the features needed to predict software defects are selected [15]. Fig. 2 shows the procedure for classification methods. Forward selection and backward selection methods have been used in [17]. Forward selection starts with the first feature, while greedy forward selection starts with the first feature, which helps to improve performance accuracy [18].

Combined learning is an effective approach to machine learning [19]. In this type of learning, to improve the accuracy of learning, the results of several classifiers are combined. There are two scenarios for creating blended learning. The first scenario is that there are different classification algorithms for defect prediction problems. A simple initial approach is to search for the best classification algorithm. However, with this method, it is not possible to utilize the additional information of other algorithms. In addition, it is difficult to determine the best classification algorithm. The second scenario is that many problems contain large and various features, and it is not appropriate to use all of them in a single class. An important issue in the construction of the blended classifier is the selection of the base classification rule. If the appropriate classifiers are not used, the variety of classifiers does not improve the efficiency of the blended classifier.

Ineffective features within the training dataset can often lead to overfitting in ML algorithms and undermine a model's ability to generalize, potentially diminishing overall classifier accuracy. The elimination of ineffective attributes enhances the performance and accuracy of the created software defect predictor. Lack of feature selection and insufficient accuracy are the main drawbacks of the majority of the previous methods, such as [25,26]. The inefficiency of the explited feature selection algorithm in finding the optimal feature due to local optima and a lower success rate is the other drawback of feature selection based methods such as [28–30]. Higher computation costs and lower stability are the other feature selection methods that were addressed in this study. The number of most effective features discovered by the feature selector algorithm is the other issue in this field that should be addressed.

3. The proposed method

3.1. Feature selection-based fault prediction

In this section, the proposed software fault predictor using the developed binary gray wolf optimizer (bGWO) and machine learning (ML) algorithms has been explained. The proposed bGWO was used to select the most effective features of the fault prediction datasets for machine learning algorithms. The ML algorithms used in this paper are artificial neural networks (ANN), decision tree (DT), K-nearest neighbor (KNN), Naïve Bayes (NB), and support vector machine (SVM). Fig. 3 shows the process of the proposed method. The standard datasets were used to train ML algorithms and to test the created software fault predictors. Thus, the bGWO selects the features that help to improve the accuracy and precision of the created fault predictor (classifier).

In the proposed bGWO, each wolf was specified as a binary feature vector with length 21 (the number of features in the training dataset) from F0 to F20. Each of the bits of this binary vector (array) corresponds to a feature in the training matrix (dataset), which is shown in Fig. 4. If the bit corresponding to each feature is zero, that feature is removed, and if it is one, that feature is included in the training. After running one round of bGWO with a random feature, the fitness value is generated for each binary feature vector. Fitness is a function of the amount of training error and the number of the selected features that we want to minimize in each round. Accordingly, each time a new set of features is selected, we gradually move toward the lowest training error in the algorithm.

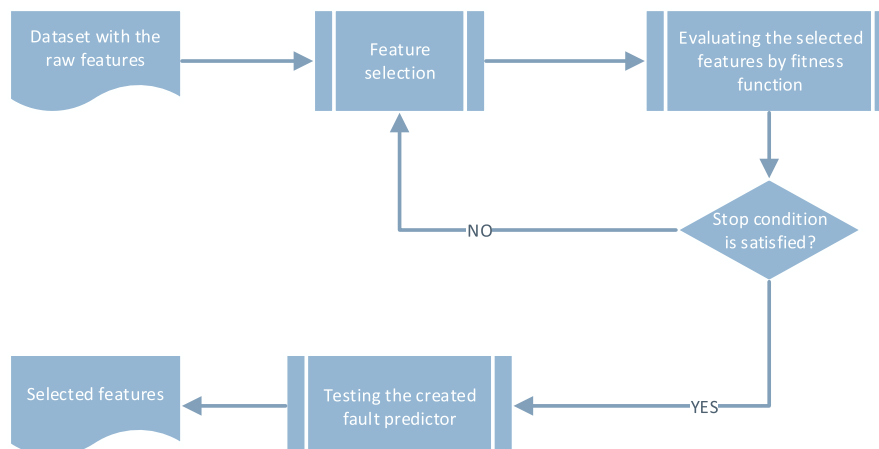


Fig. 2. The procedure for the classification methods for feature selection.

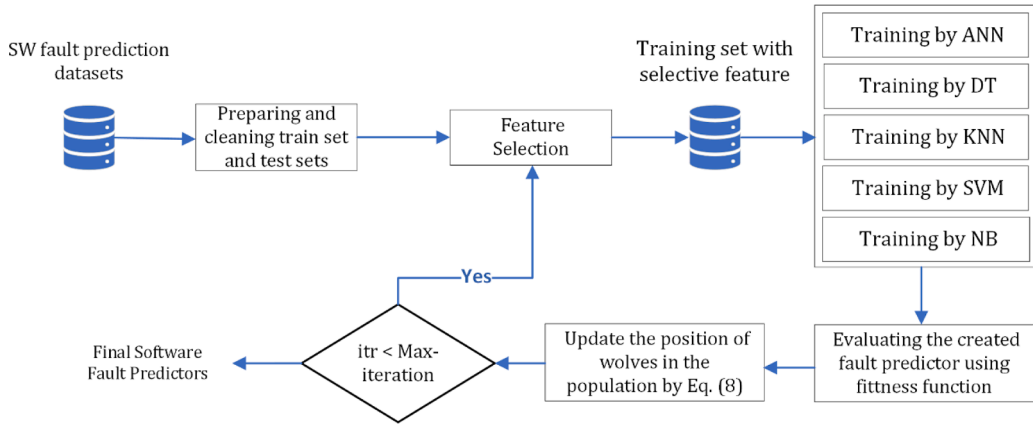


Fig. 3. The process of the proposed method for software fault prediction.

3.2. Training dataset

The required datasets have been used from the NASA repository [20], which has been available to the public since 2005. The software criteria used from this dataset are McCabe's criterion, Halstead's criterion, branch count, and five different criteria indicating lines of code. Table 2 explains the datasets used in this study and Table 3 illustrates the features of the datasets.

3.3. Selection of effective features

3.3.1. Structure of gray wolf optimizer

After creating the preparing the training dataset, in the second stage, a binary version of the GWO algorithm was developed. The developed bGWO algorithm was used for feature selection. Indeed, the optimal features of the training dataset were selected before creating the desired classification model by the machine learning algorithms. The gray wolf optimization algorithm is a metaheuristic algorithm inspired by the hierarchical structure and social behavior of gray wolves during hunting [21]. In GWO, each wolf is implemented by a vector of length 21 (the number of features). In the wolf vector, each of the cells is initially set to 1, which means that all the features are included in the training. A population matrix includes n rows, and each row includes 21 columns. In each group of wolves to hunt, there are four levels, which are modeled as a pyramidal structure: the leader wolves are called the alpha (α) group, which can be male or female. Beta (β) wolves are assisted by alpha wolves in the decision-making process and are also susceptible to being selected instead. Delta (δ) wolves are lower than beta wolves and include old predators. Omega (ω) wolves have the lowest rank in the hierarchy pyramid, which is the least right than the rest of the group; they eat and are not involved in the decision-making process. GWO algorithm consists of three main steps: Tracking and Approaching, encircling, and attacking. In the GWO, Eqs. (1) and (2) were used to mathematically model the encircling behavior of the gray wolves.

$$\vec{D} = \left| \vec{C} \cdot \vec{X}_p(t) - \vec{X}(t) \right| \quad (1)$$

$$\vec{X}(t+1) = \vec{X}_p(t) - \vec{A} \cdot \vec{D} \quad (2)$$

In Eqs. (1) and (2), t represents the current repetition, and $\vec{A}$ and $\vec{C}$ the coefficient vectors, and $\vec{X}$ the position vector of a gray wolf, the $\vec{A}$ and $\vec{C}$ vectors are calculated by Eqs. (3) and (4).

$$\vec{A} = 2\vec{a} \cdot \vec{r}_1 - \vec{a} \quad (3)$$

$$\vec{C} = 2 \cdot \vec{r}_2 \quad (4)$$

In Eqs. (3) and (4), $\vec{a}$ vector is linearly reduced from 2 to 0 during repetition and $\vec{r}_1$ and $\vec{r}_2$ are random vectors in the interval [0,

F0	F1	F2	F3		F20	F21
1	0	1	1	...	0	1

Fig. 4. The structure of a wolf in the bGWO as a binary vector to specify features.

Table 1
Comparison of the software fault prediction methods.

Paper	Description	Used ML techniques	Results	Disadvantages
[25]	Failure prediction using several ML techniques	Decision tree, bagging, random forest, gradient boosting	Combination of different ML algorithms leads to the overall prediction performance.	Lack of feature selection and insufficient accuracy
[26]	Fault prediction using the linear homogeneous ensemble	Differentiable Extreme Learning Machine Ensemble and Non-differentiable Extreme Learning Machine Ensemble	Consistent performance for most of used datasets	Lack of feature selection and insufficient accuracy
[27]	Software fault prediction for the imbalanced datasets	Combination of wagging and AdaBoost	Higher and stable performance than the AdaBoost	The unique features of the datasets were not selected and considered in the training
[28]	Combination of Particle swarm optimization (PSO) and ensemble methods for the software fault prediction	PSO and bagging ensemble methods	Combination of PSO and ML methods outperforms the other classifiers for the software fault prediction	The probability of the local optimum in the PSO causes the optimal features not to be selected
[29]	Combination of meta-heuristic algorithm and ensemble ML algorithms for the fault prediction	Harmony search-based optimization algorithm and cost-sensitive boosting	Combination of the meta-heuristic algorithm (for parameter tuning) and boosting improves the performance of the created fault predictor	Inefficiency of the harmony search algorithm leads to insufficient accuracy
[30]	An analysis of ensemble ML techniques with feature selection to predict software fault	Different feature ranking techniques and their combination with the Naive Bayes algorithm	Combination of different feature ranking with the Naive Bayes algorithms improves the performance of the fault predictors	Inefficiency of the used feature ranking against heuristic-based feature selectors in the fault prediction datasets
[31]	Combination of the salp swarm algorithm (SSA) combined with a backpropagation neural network (BPNN) to create software fault predictor	SSA was used as feature selector and BPNN was used as classification algorithm	The combination of the SSA with BPNN leads to higher prediction accuracy.	The high computational cost in the majority of data sets
[32]	Two-stage classification method and a two-stage ranking method were used to create fault predictor	Three-way decisions and ensemble learning to predict software defect.	This method provides a higher accuracy and a lower decision cost.	Converting the three-way decisions into two-way decisions is the main challenge of this method

Table 2
Specification of the NASA datasets used in this study.

Number	Project Name	Programming Language	Number of Models	Percentage of defected models (%)	Project description
1	CM1	C	498 (349 Train, 149 Test)	9.7	NASA spacecraft instrument
2	JM1	C	10,885 (7619 Train, 3266 Test)	19	a real-time predictive ground system which uses simulations to generate predictions
3	KC1	C++	2109 (1476 Train, 633 Test)	15.4	system implementing storage management for receiving and processing ground data
4	KC2	C++	522 (365 Train, 157 Test)	20	Science data processing
5	PC1	C	1109 (776 Train, 333 Test)	6.8	flight software for earth orbiting satellite

1]. A gray wolf with a location (X, Y) can update its location according to the position of the prey (X^*, Y^*) . The locations of the gray wolves were updated based on the (X^*, Y^*) . The Algorithm iteratively converges to the best solution (location of prey). The best solution indicates the minimum number of features of the dataset that have the maximum effect on the software fault prediction. Gray wolves can detect the location of the prey and surround them. The alpha wolves are often the ones that hunt; however, beta and delta wolves occasionally join them. Alpha, beta, and delta wolves are more aware of the probable location of prey; their locations are updated by Eqs. (5)–(7). This allows us to mathematically replicate the hunting behavior of gray wolves. Fig. 5 illustrates how a search agent updates its position according to Alpha, Beta, and Delta in the two-dimensional search space. The Pseudo-code of the GWO algorithm is presented in Algorithm 1.

Table 3

Description of features used in NASA datasets.

Number	Feature	Description
1	LOC	line count of code (McCabe)
2	v(g)	Cyclomatic complexity (McCabe)
3	eV(g)	Essential complexity (McCabe)
4	iv(g)	Design complexity (McCabe)
5	N	Operators + Operands (McCabe)
6	v	Volume (Halstead)
7	L	Program length (Halstead)
8	D	Difficulty (Halstead)
9	I	Intelligence (Halstead)
10	B	Effort: Halstead effort to write program
11	E	Number of Delivered Bugs (Halstead)
12	T	Time to write program (Halstead)
13	LOCode	Line count (Halstead)
14	LOComment	Count of lines of comments (Halstead)
15	LOBlank	Count of blank lines (Halstead)
16	LOCodeAndComment	Count of lines which contain both code and comments (Halstead)
17	uniq_Op	unique operators (Halstead)
18	uniq_Opnd	unique operands (Halstead)
19	total_Op	Total operators (Halstead)
20	total_Opnd	Total operands (Halstead)
21	branchCount	Number of branches in flow graph (Halstead)
22	defects	Number of defects

$$\vec{D}_\alpha = \left| \vec{C}_1 \cdot \vec{X}_\alpha - \vec{X} \right|, \vec{D}_\beta = \left| \vec{C}_2 \cdot \vec{X}_\beta - \vec{X} \right|, \vec{D}_\delta = \left| \vec{C}_3 \cdot \vec{X}_\delta - \vec{X} \right| \quad (5)$$

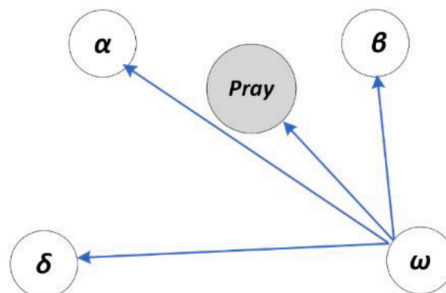
$$\vec{X}_1 = \vec{X}_\alpha - \vec{A}_1 \cdot \left(\vec{D}_\alpha \right), \vec{X}_2 = \vec{X}_\beta - \vec{A}_2 \cdot \left(\vec{D}_\beta \right), \vec{X}_3 = \vec{X}_\delta - \vec{A}_3 \cdot \left(\vec{D}_\delta \right) \quad (6)$$

$$\vec{X}(t+1) = \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3} \quad (7)$$

3.3.2. Binary gray wolf optimizer

As shown in Fig. 3, the developed bGWO is used to select effective features for machine learning algorithms. The Continuous Gray Wolves Optimization (OGWO) algorithm constantly shifts their positions to anywhere in the state space. In the feature selection problem, solutions are limited to binary values of zero and one which creates a binary version using sigmoid function [33]. In the method, the wolf Equation is to update a function of three positional vectors, namely, $\vec{X}_\alpha$, $\vec{X}_\beta$ and $\vec{X}_\delta$. It attracts each wolf to the first of the top three solutions. The set of solutions at any given time is binary. In BGWO, Eq. (7) can be formulated as Eq. (8). In Eq. (8), $\text{Crossover}(\vec{x}_1, \vec{x}_2, \vec{x}_3)$ is a suitable shift between x, y and z solutions. The binary vectors $\vec{x}_1$, $\vec{x}_2$ and $\vec{x}_3$ effectively move wolves toward alpha, beta, and delta wolves. The vectors $\vec{x}_1$, $\vec{x}_2$ and $\vec{x}_3$ are calculated using Eq. (9).

$$\vec{X}_i^{t+1} = \text{Crossover}(\vec{x}_1, \vec{x}_2, \vec{x}_3) \quad (8)$$

**Fig. 5.** Updating positions of wolves in GWO.

Algorithm 1

The abstract algorithm of the GWO [21].

```

GWO initialization;
Calculate the wolves fitness;
 $X_\alpha = \alpha$  position;
 $X_\beta = \beta$  position;
 $X_\delta = \delta$  position;
Iter = 0;
While (iter < max)
{
    foreach wolf do
        Modify wolves' position using Eqs. (3-7)
    update (A);
    update(C);
    update(a);
    Calculate the wolves fitness;
    Update the positions of  $\alpha$ ,  $\beta$ ,  $\delta$ ;
    iter=iter+1;
}
return ( $X_\alpha$ );

```

$$\vec{X}_1^d = \begin{cases} 1 & \text{if } (X_\alpha^d + bstep_\alpha^d) \geq 1 \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

So, the vector $\vec{X}_1^d$ is the position of the alpha wolf and $bstep_\alpha^d$ a binary step in the d dimension. $bstep_\alpha^d$ is calculated based on Eq. (10). Here, $rand$ is a random number based on a uniform distribution between zero and one. $cstep_\alpha^d$ is the step size being a continuous value for d dimension and is calculated using the Sigmoid function (Eq. (11)).

$$bstep_\alpha^d = \begin{cases} 1 & \text{if } (X_\alpha^d + cstep_\alpha^d) \geq rand \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

$$cstep_\alpha^d = \frac{1}{1 + e^{-10(A_1^d D_\alpha^d - 0.5)}} \quad (11)$$

To calculate X_2^d and X_3^d which are the positions of beta and delta wolves, respectively, Eq. (12) is used. So, the $\vec{X}_2^d$ vector positions are beta wolves and $bstep_\beta^d$ a binary step in the d dimension. $bstep_\beta^d$ can be calculated based on Eq. (13).

$$\vec{X}_2^d = \begin{cases} 1 & \text{if } (X_\beta^d + bstep_\beta^d) \geq 1 \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

$$bstep_\beta^d = \begin{cases} 1 & \text{if } (X_\beta^d + cstep_\beta^d) \geq rand \\ 0 & \text{otherwise} \end{cases} \quad (13)$$

In Eqs. (10) and (13), $rand$ is a random number based on a uniform distribution between zero and one. $cstep_\beta^d$ is the step size being the continuous value for the d dimension. It is calculated using the Sigmoid function indicated by Eq. (14). Here, also, A_2^d and D_β^d respectively using Eqs. (1) and (3). So that the $\vec{X}_3^d$ is vector positions of delta wolf (calculated by Eq. (15)) and $bstep_\delta^d$ are a binary step in the d dimension. $bstep_\delta^d$ is calculated based on Eq. (16).

$$cstep_\beta^d = \frac{1}{1 + e^{-10(A_2^d D_\beta^d - 0.5)}} \quad (14)$$

$$\vec{X}_3^d = \begin{cases} 1 & \text{if } (X_\delta^d + bstep_\delta^d) \geq 1 \\ 0 & \text{otherwise} \end{cases} \quad (15)$$

$$bstep_\delta^d = \begin{cases} 1 & \text{if } (X_\delta^d + cstep_\delta^d) \geq rand \\ 0 & \text{otherwise} \end{cases} \quad (16)$$

In Eq. (16), a random number $rand$ is based on a uniform distribution between zero and one and $cstep_\delta^d$ the step size is a continuous value for the d dimension and is calculated using the Sigmoid function (Eq. (17)). In this regard, also, A_3^d and D_δ^d are calculated using Eqs. (1) and (3). X_d is calculated by Eq. (18). In this relationship, a_d , b_d and c_d are binary values for the first, second, and third

parameters in dimension d . The X_d is the output of Crossover in dimension d and $rand$ is a random number from a uniform distribution in the range of zero and one. The pseudocode of the bGWO1 is explained in [Algorithm 2](#).

$$cstep_o^d = \frac{1}{1 + e^{-10(A_3^d D_o^d - 0.5)}} \quad (17)$$

$$X_d = \begin{cases} a_d & \text{if } rand < \frac{1}{3} \\ b_d & \text{if } \frac{1}{3} \leq rand \leq \frac{2}{3} \\ c_d & \text{otherwise} \end{cases} \quad (18)$$

To obtain the fitness of feature vectors, [Eqs. \(19\) and \(20\)](#) were suggested; in which the fitness of a solution (selected features) is defined as the sum of the training error and the number of features. Fitness is a minimization function. In [Eq. \(19\)](#), the training error is a number between zero and one, which is obtained from the difference of accuracy from 100 and normalized to the range [0, 1]. Relation (3.2) shows the training error. The number of features in [Eq. \(19\)](#) represents the number of selected features for training which is normalized into the range [0, 1].

$$Fitness(i) = ErrorPercentageTest(i) + UsedFeatures \quad (19)$$

$$ErrorPercentageTest(i) = (100 - Accuracy(i))/100 \quad (20)$$

4. Experiments and results

4.1. Experiments platform and datasets

This section evaluates the results of the proposed method for predicting software defects. First, we obtain the results of machine learning algorithms (neural networks, decision trees, K-nearest neighbors, Naïve Bayes, and support vector machine) in a normal way, i.e., without using effective features. Next, we obtain the results of the proposed method using bGWO which is used to select the effective features. Then the results of both approaches are compared. The proposed method was implemented along with ant colony optimizer (ACO) using MATLAB version 2022, on Windows 10 with Core i7 Intel, 2 GHz, and 4 GB RAM. [Table 4](#) shows the calibration parameters of developed bGWO and ACO methods to select the effective features in predicting software defects. The binary version of the ACO was suggested as feature selector in [\[22\]](#).

Training and Testing stages were performed on five datasets (described in [Table 2](#)). In the first step, 70 % of the data are randomly used to train an artificial neural network, decision tree, K-nearest neighbor, Naïve Bayes, and support vector machine models. In the test phase, 30 % of the data that were not used in the training phase were used to test learning models. [Table 5](#) shows the features of test and training data. The confusion matrix is defined as [Table 6](#); according to this table, the values of sensitivity, specificity, and efficiency can be calculated.

4.2. Evaluation criteria

In this study, accuracy, precision, recall, and F1 are used as evaluation criteria in software modules classification problems; [Eqs. \(21\)-\(24\)](#) were used to calculate these criteria.

Algorithm 2

The abstract algorithm of the binary GWO.

```

n: number of wolves;
Initialize the wolves' position randomly with binary values;
Find the best wolves as  $\alpha$ ,  $\beta$  and  $\delta$ ;
while (stopping criterion)
{
    foreach wolf in the population do
    {
        Calculate x1, x2 and x3 by Eqs. \(8-12\);
         $x_i^{t+1} = \text{crossover}(x1, x2, x3)$ ;
    }
    Update (a);
    Update (C);
    Calculate the fitness of wolves;
    Find the best wolves as  $\alpha$ ,  $\beta$  and  $\delta$ ;
}
return ( $X_a$ );

```

Table 4

The bGWO parameters' value to select the effective features.

Algorithms	Parameter Name	Value
Gray Wolf Algorithm (GWA)	Number of wolves	30
	a	Random value from [0, 2]
	C	Determined by GWO movement relations
	A	Determined by GWO movement relations
	r1, r2	[0–1]
Ant Colony optimization algorithm (ACO)	Number of iterations	100
	Number of Nests	30
	Lavy Distribution Parameter	1.5
	Step Length	0.01
	Number of iterations	100

Table 5

Features of the used test and training data.

Dataset	Total number of samples	Number of train samples	Number of test samples
CM1	496	347	149
JM1	10,885	7619	3266
KC1	2107	1474	633
KC2	523	366	157
PC1	1107	775	333

Table 6

Confusion matrix.

Model prediction		
Yes	No	
TP	FN	Real Yes (A)
FP	TN	Real No (B)

$$\text{Accuracy} = \frac{\text{True Positive} + \text{True Negative}}{\text{All test samples}} \quad (21)$$

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}} \quad (22)$$

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}} \quad (23)$$

$$F_1 = \frac{2 * (\text{Precision} * \text{Recall})}{\text{Precision} + \text{Recall}} \quad (24)$$

In general, the confusion matrix was used to check the success rate and efficiency of classification models. Confusion matrix analysis in classification and prediction leads to four modes: true positive (TP), true negative (TN), false positive (FP), and false negative (FN).

- TP: Number of positive samples that are correctly classified into the positive class.
- FP: Number of negative samples that are wrongly classified into the positive class.
- FN: Number of positive samples that are wrongly classified into the negative class.
- TN: Number of negative samples that are correctly classified into the negative class.

Three criteria of precision, accuracy, and efficiency are obtained from the confusion matrix; these criteria are used to analyze the performance of software module classification models. An extensive series of experiments have been performed on the classifiers created by different ML algorithms (artificial neural network, decision tree, K-nearest neighbor, Naïve Bayes, and support vector machine). The experiments have been performed to respond to the following RQs on each dataset:

- **RQ1:** Does the proposed method increase the accuracy, precision, recall, and F1 of the ML algorithms in the software fault prediction problem?
- **RQ2:** Does the proposed method identify and eliminate the ineffective features of the training datasets?
- **RQ3:** Is the convergence speed and success rate of the proposed bGWO higher than the other methods?

4.3. Discussion of the first experiment's results (CM1 dataset)

The first experiment has been performed on the CM1 data set as a training and test data set to respond to RQ1. The results of running the machine learning algorithm on the CM1 dataset to determine the performance criteria for artificial neural network (ANN), decision tree (DT), K-nearest neighbor (KNN), Naïve Bayes (NB), and support vector machine (SVM) with and without applying the proposed bGWO (feature selector) are shown in Fig. 6; in this Figure, the created classifiers by different ML algorithms (ANN, DT, KNN, NB and SVM) have been compared in terms of accuracy, precision, recall and F1. The first series of experiments have been performed on the raw train set. In the raw train set, all features are taken into account. Redundant and irrelevant features in the training dataset can result in overfitting in the ML algorithms. The goal of feature selection is to choose a limited group of features that are enough to accurately predict the target labels. Redundant features may deceive ML algorithms. The feature selection methods also save costs by reducing unimportant information. The second series of experiments has been performed on each dataset using the bGWO and ML algorithms. The bGWO has been used to find the optimal features of the train sets. The results of the implemented ML algorithm on the CM1 dataset using the proposed bGWO to determine the effective features are shown in Fig. 6.

By comparing the results, it can be concluded that the efficiency criteria for predicting software defects through the selected features using bGWO are better than the previous method. The accuracy and precision of the classifier created by ANN without using bGWO are 88.15 % and 88.46 %, respectively; meanwhile, these figures were increased to 91.27 % and 95.07 % while bGWO was used as the feature selection. The results of experiments indicate that the use of bGWO improves the performance of the ML algorithms in the software fault prediction problem. Thanks to the use of the selected effective features (by bGWO), ML+bGWO has higher performance than pure ML algorithms. In sum, the proposed method of bGWO for selecting effective features has acceptable performance on the ML algorithms using the CM1 dataset.

The average performance of the classifiers created by ML algorithms with and without feature selection is shown in Fig. 7. The average accuracy of the classifiers created by different ML algorithms (ANN, DT, KNN, NB, and SVM) without the feature selection method is 87.02 %, whereas this figure is 92.61 % when the same ML algorithms are executed using the proposed feature selection method. Similar results have been obtained for the other performance criteria. The average precision of the classifiers created by ML algorithms without and with the proposed feature selection algorithm is about 92.63 % and 93.61 %, respectively. Regarding the results, the average value of the recall metric for the classifiers created by the proposed method is considerably higher than the pure ML algorithms. Finally, the classifiers created by the proposed method are more effective than the classifiers created by ML algorithms without feature selection.

The other important criterion in the feature selection-based fault prediction methods is the number of selected features by the bGWO algorithm. Ineffective features weaken the model's capacity for generalization and may also lower a classifier's overall accuracy. Moreover, a model's total complexity rises with the number of variables added. The proposed feature selection algorithm finds ineffective features and consequently improves fault prediction accuracy. Fig. 8 shows the number of effective features found by the proposed bGWO in different ML algorithms. CM1, as the real-world fault prediction dataset, includes 21 independent and one dependent feature. The proposed bGWO selects a small subset of the features as the most effective features in the CM1 dataset.

The proposed method's average performance was calculated for 10 times execution and, the proposed method shows a good performance for detection. Among the 21 features, 2, 3, 4, 7, 8, 13, and 21 features were selected by the bGWO in different ML algorithms. Fig. 9 compares the average number of selected features by the bGWO and binary ACO during 10 executions. As shown in Fig. 9, the same number of features were selected by bGWO and ACO when the ANN was used as a classification algorithm. In the DT classification algorithm, the number of selected features by bGWO and ACO are 5 and 7 respectively; these are Figs. 9 and 15 for the KNN classification algorithm. Regarding the results, the number of selected features by the bGWO is lower than the number of selected features by ACO. Indeed, bGWO has higher performance than the ACO in finding the most effective features.

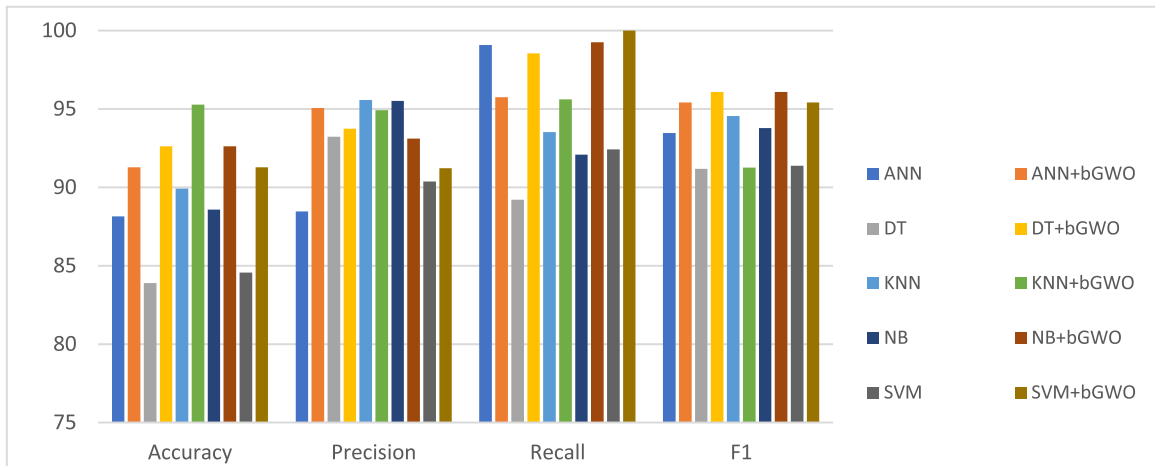


Fig. 6. The effects of the developed bGWO on the performance of the ML algorithms in CM1 dataset.

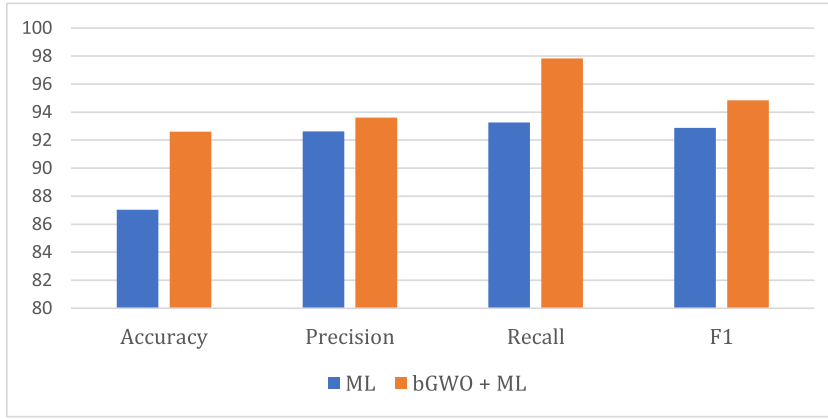


Fig. 7. The average values of the performance criteria of ML (ANN, DT, KNN, and NB) algorithms with and without bGWO as the feature selection in the CM1 dataset.

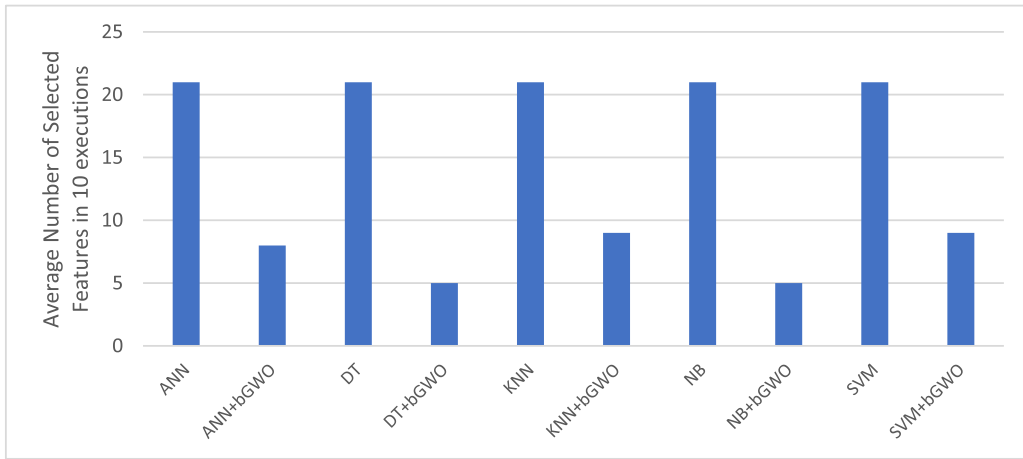


Fig. 8. The average number of selected features by bGWO in different ML algorithms in CM1 dataset in ten times executions.

The selected features of the CM1 dataset by the bGWO in different ML algorithms in the best results from 10 executions are shown in Table 7. To this end, each ML algorithm with the feature selectors has been executed 10 times. The selected features in the best executions (highest evaluation criteria) were considered as the most effective features. Table 7 indicates the most effective features that have been selected by different algorithms. Regarding the results, different features have been chosen by different methods. Fig. 10 shows the selection rate of different features of the CM1 data set by the bGWO during 10 executions. Regarding the results, the most effective features identified by the bGWO in the CM1 dataset are as follows respectively:

1. Halstead difficulty
2. Halstead program length
3. McCabe essential complexity
4. Halstead's line count
5. McCabe cyclomatic complexity
6. McCabe design complexity
7. Halstead total operators + operands
8. Halstead volume
9. Branch Count of the flow graph
10. Halstead effort

To show the stable state of running the proposed method on the CM1 dataset, it has been executed 10 times and the results have been recorded. The results show an improvement in accuracy and precision measures. Considering 21 features, the proposed algorithm applies effective features for training and testing, which in turn reduces the execution time of the learning algorithm and increases its efficiency. Some features have been selected many times in the training by bGWO, which indicates that these diagnostic features are

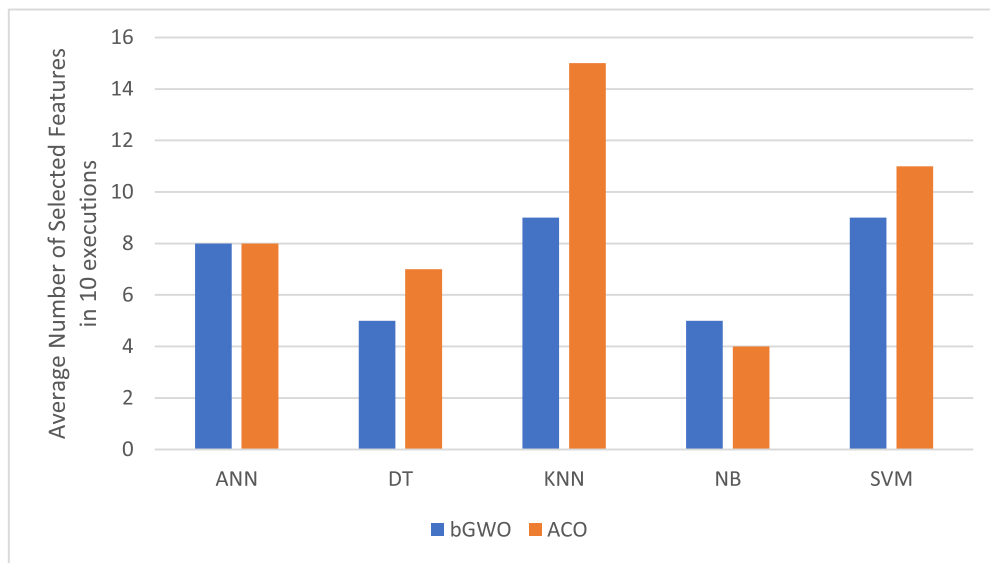


Fig. 9. The average number of selected features by ACO and bGWO in different ML algorithms in the CM1 dataset during ten times executions.

Table 7

The selected features by bGWO and ACO in different ML algorithms in CM1 in the best result from 10 executions.

	ANN		DT		KNN		NB		SVM	
	bGWO	ACO	bGWO	ACO	bGWO	ACO	bGWO	ACO	bGWO	ACO
1	1	1	0	0	0	1	0	1	0	0
2	0	1	0	0	0	0	0	0	0	1
3	1	1	0	0	1	1	0	0	1	1
4	0	1	0	1	1	0	1	1	0	1
5	1	0	0	0	1	1	0	0	0	0
6	1	1	0	0	1	1	0	0	0	0
7	1	0	1	1	1	1	0	1	1	0
8	1	0	1	0	1	1	1	0	1	1
9	1	0	0	0	0	0	0	0	0	0
10	0	0	0	0	1	1	0	1	1	1
11	0	0	0	0	0	0	1	0	0	0
12	1	1	1	1	1	0	1	0	0	0
13	0	1	0	1	0	1	1	0	1	1
14	0	0	0	0	1	0	0	0	1	1
15	0	0	0	0	0	1	0	0	1	1
16	0	0	0	1	0	1	1	0	1	1
17	0	0	0	1	0	1	0	0	1	1
18	0	0	1	0	0	1	0	0	0	1
19	0	0	0	0	0	0	0	0	1	0
20	0	1	1	1	0	1	0	0	0	1
21	1	0	1	0	0	0	0	1	1	0

more effective than others. Finally, Fig. 11 shows the convergence of the proposed bGWO in finding the most effective features of the CM1 dataset. bGWO has different performances in different ML algorithms. As shown in Fig. 11, the bGWO attains its best performance (minimum fitness value and maximum convergence speed). After ANN, the KNN and NB have higher performance than the other algorithms in the software fault prediction methods. Overall, the proposed bGWO (as the feature selector) leads to considerable performance enhancement in the ML algorithm in the software fault prediction problem.

4.4. Discussion of the second experiment's results (KC1 dataset)

The second series of experiments has been conducted to respond to the RQs using the KC1 dataset. The second experiment was run using the KC1 data set as the training and test data sets. This dataset indicates the features of a real-world software application. KC1 includes 2107 records and 21 features. 1475 records were considered a train set, and 633 records were considered a test data set. The performance metrics of the improved ML algorithms using bGWO (a feature selection technique) and the traditional ML methods on the KC1 dataset are contrasted in Fig. 12. In terms of outcomes, the traditional ML approach is outperformed by the ML algorithms on

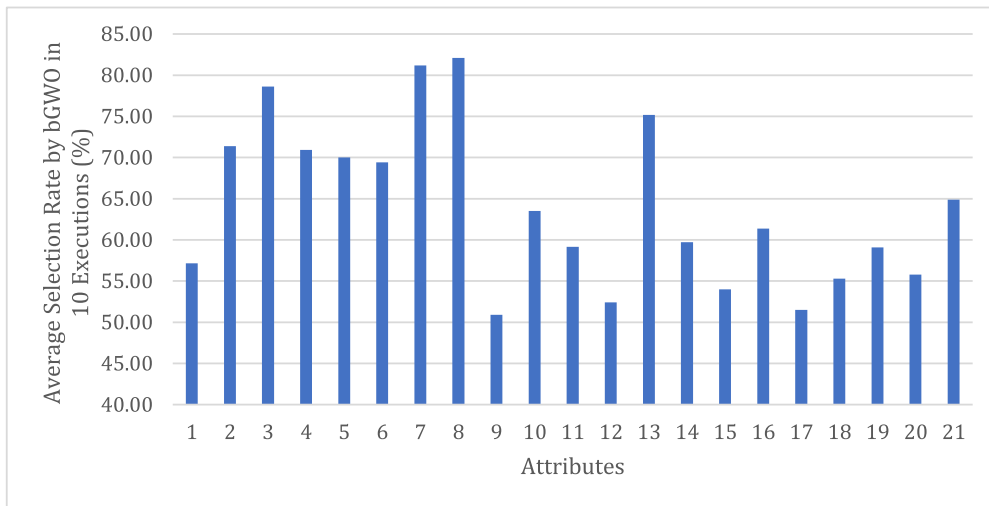


Fig. 10. The selection rate of the 21 features by the bGWO in the CM1 dataset during 10 executions.

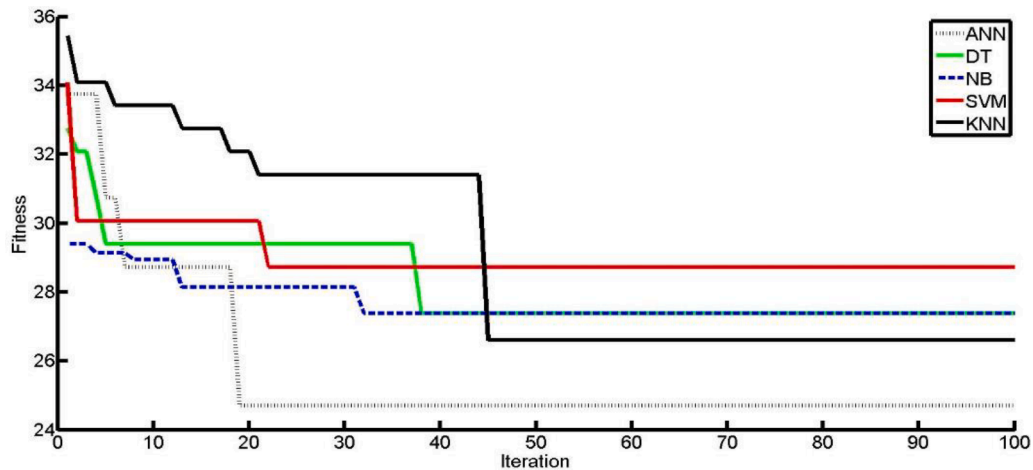


Fig. 11. Convergence of the bGWO for finding effective features with different ML algorithms in the CM1 dataset.

the training dataset with selected features. Fig. 12 compares the performance measures of the two conventional and optimized ML models on the KC1 dataset. The classifiers created by the various ML algorithms have been compared in terms of accuracy, precision, recall, and F1. The raw train set has been used for the first round of trials. All features have been taken into consideration in the raw train set. The classifier created by KNN using the filtered train set by bGWO has the highest accuracy and precision. The fault predictor created by the ANN outperforms the other predicting models in terms of recall and F1.

Overfitting in machine learning algorithms can be caused by unnecessary features in the training dataset. ML algorithms may be tricked by redundant features. KC1 is the other dataset that was used to evaluate the effect of the performance of the ML algorithms. Using the bGWO and ML algorithms, the second round of tests has been carried out on the KC1 dataset. The classifier built using ANN without the use of bGWO has an accuracy and precision of 84.65 % and 85.01 %, respectively; however, when bGWO was employed as the feature selection, these numbers rose to 86.25 % and 87.52 %, respectively. The tests show that using bGWO enhances the ML algorithms' performance in the software failure prediction problem in the KC1 dataset. By analyzing the results shown in Fig. 12, we can conclude that the efficiency criteria for predicting software defects through the selection of effective features using bGWO are better than the previous method. Regarding the results, ML+bGWO performs better because it takes advantage of the chosen effective features. The suggested bGWO technique for choosing useful features performs satisfactorily on the ML algorithms employing the KC1 dataset. The accuracy, precision, recall, and F1 of the optimized ML algorithms with the bGWO method are higher than all other models. In sum, the proposed bGWO for selecting the effective features has acceptable performance on the KC1 dataset.

Fig. 13 displays the average performance of the classifiers produced by ML algorithms, both with and without feature selection, on the KC1 dataset. When the suggested feature selection strategy is applied to the ML algorithms (ANN, DT, KNN, NB, and SVM), the average accuracy of the classifiers produced by the ML and pure ML+bGWO algorithms is 84.59 % and 87.75 %, respectively. The

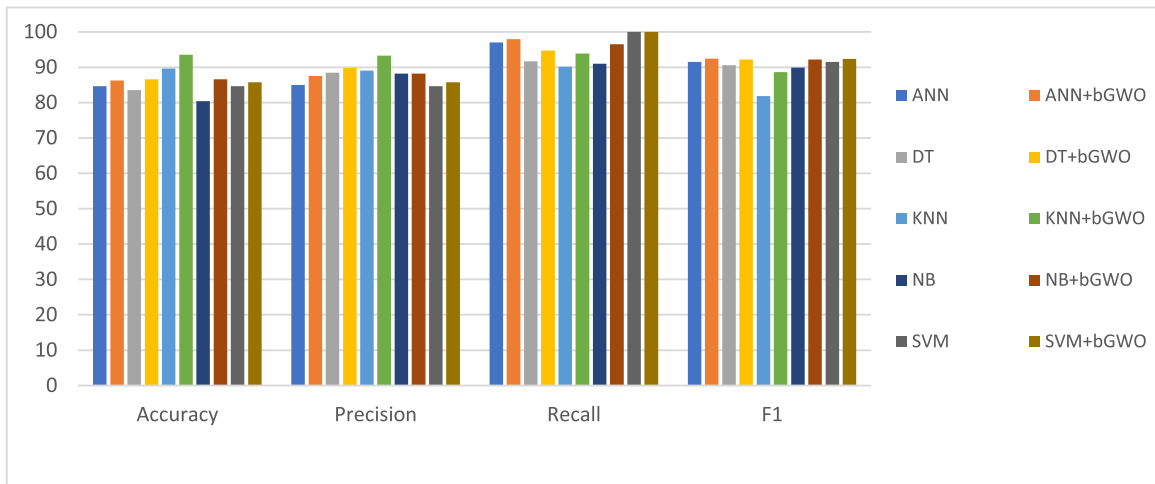


Fig. 12. The effects of the developed bGWO on the performance of the ML algorithms in the KC1 dataset.

outcomes for the other performance criteria are comparable. The classifiers produced by ML algorithms without (ML) and with the suggested feature selector (ML+bGWO) have an average precision of around 87.05 % and 88.90 %, respectively. In terms of outcomes, the classifiers developed using the suggested technique have an average recall metric value that is significantly greater than that of the pure ML algorithms. In the end, the classifiers produced by the suggested technique outperform those produced by ML algorithms without feature selection. The suggested bGWO increases the fault prediction models' accuracy by identifying and removing inefficient features. The number of useful features discovered by the suggested bGWO in various ML algorithms is displayed in Fig. 14. The KC1 dataset's most useful features are chosen by the suggested bGWO from a restricted selection of features.

After ten times execution, the average performance of the suggested approach was computed on KC1, and it demonstrated good detection performance. The average number of features chosen by the binary ACO and bGWO across ten executions in the KC1 dataset is shown in Fig. 15. As can be seen in Fig. 15, when the ANN was utilized as the classification algorithm, bGWO and ACO chose 7 and 16 features respectively. The number of features chosen by bGWO and ACO in the DT classification algorithm is 8 and 12, respectively; for the KNN classification algorithm, these numbers are 8 and 9. The number of features chosen by bGWO and ACO in the NB classification algorithm are 6 and 9. All in all, the average number of selected features by the proposed bGWO is considerably lower than the binary ACO.

Table 8 displays the KC1 dataset features that the bGWO chose and optimized using several ML algorithms, yielding the best results after 10 runs. Each ML method has been run ten times. The most effective features were those that were chosen for the best implementations (highest assessment criteria). Table 8 presents the most efficient features derived from several techniques in its best execution. The selection rate of features in the KC1 data set across ten executions by the bGWO is displayed in Fig. 16. In light of the findings, the most effective features that have been selected by bGWO's in the KC1 dataset are as follows:

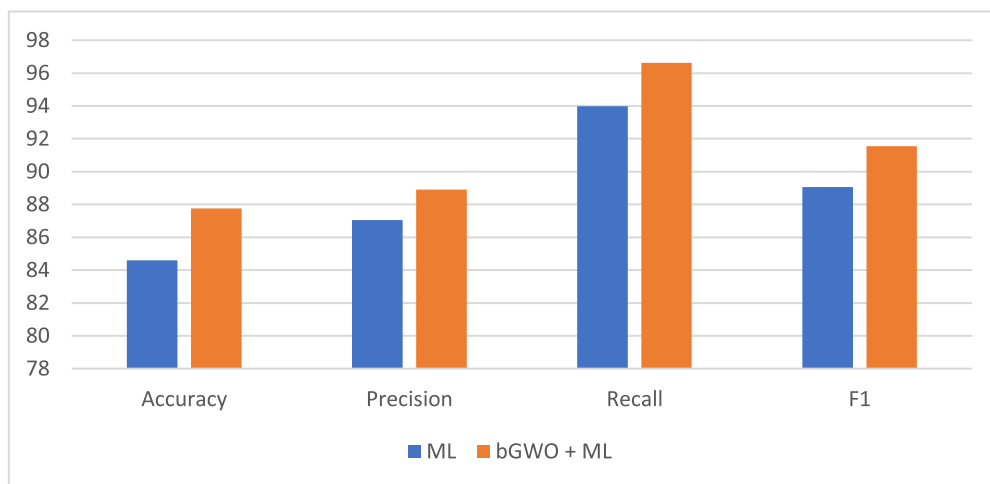


Fig. 13. The average values of the performance criteria of ML (ANN, DT, KNN, and NB) algorithms with and without bGWO as the feature selection in the KC1 dataset.

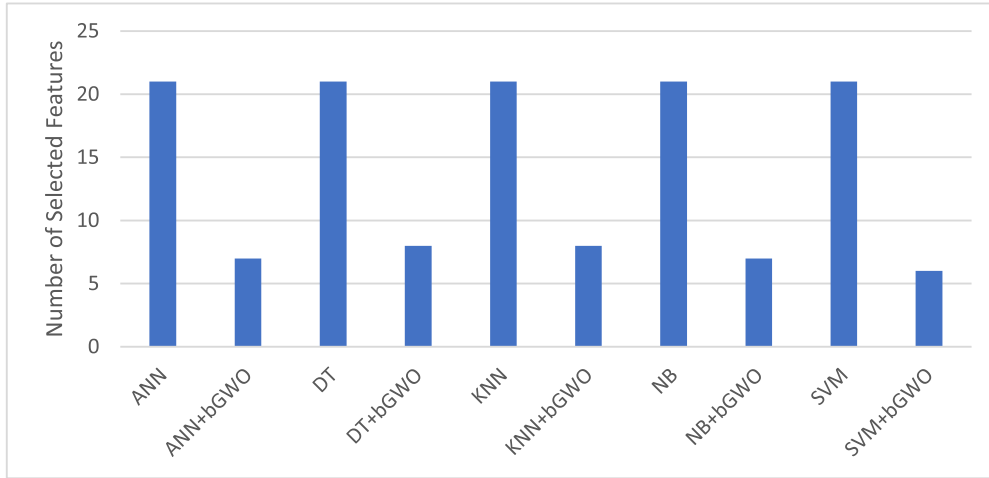


Fig. 14. The average number of selected features by bGWO in different ML algorithms in the KC1 dataset in ten times executions.

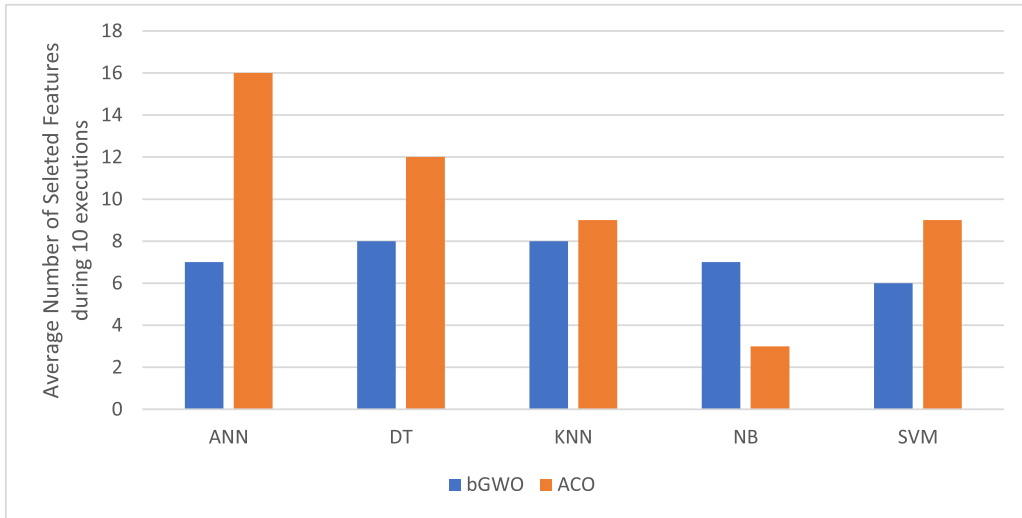


Fig. 15. The average number of selected features by ACO and bGWO in different ML algorithms in the KC1 dataset during 10 executions.

1. McCabe essential complexity
2. McCabe design complexity
3. Halstead total operators + operands
4. Halstead program length
5. Halstead effort
6. Halstead "Number of Delivered Bugs"
7. Halstead's count of lines of comments
8. Line of Code and Comment
9. unique operators
10. branch Count

The suggested approach has been performed ten times on the KC1 dataset to demonstrate its stable status. The outcomes demonstrate an increase in precision and accuracy metrics. The suggested approach takes into account 21 features and uses the most effective features for training the classifier. The subset of specific features has been chosen repeatedly during bGWO's training. The selected features have the highest effect on the accuracy and precision of the created classifier. The relative contribution of each parameter to the prediction in the KC1 dataset is displayed in Fig. 16. Lastly, Fig. 17 illustrates how the suggested bGWO converges to the best results in the KC1 dataset. The bGWO performs differently across a range of ML techniques. The bGWO achieves optimal performance, as seen in Fig. 17 (minimum fitness value and maximum convergence speed). In terms of software defect prediction

Table 8

The selected features by bGWO and ACO in different ML algorithms in KC1 in the best result from 10 executions.

#	ANN		DT		KNN		NB		SVM	
	bGWO	ACO	bGWO	ACO	bGWO	ACO	bGWO	ACO	bGWO	ACO
1	1	1	0	1	1	1	0	0	0	1
2	0	1	1	1	0	1	0	0	0	1
3	0	1	1	1	1	0	1	1	1	0
4	1	1	0	0	1	1	0	1	0	0
5	1	1	1	1	1	0	1	0	1	0
6	0	1	0	0	0	1	1	0	1	1
7	0	1	1	0	1	1	0	1	0	1
8	0	0	0	1	1	1	0	0	0	1
9	1	0	0	0	1	0	0	0	0	0
10	0	1	0	1	0	0	1	1	1	0
11	1	1	0	1	1	0	1	1	1	0
12	0	1	0	1	0	0	0	0	0	1
13	0	0	1	0	0	0	0	0	0	0
14	1	0	0	0	1	1	0	0	0	0
15	0	1	0	1	0	0	1	0	1	0
16	0	1	1	1	0	1	1	1	1	1
17	1	1	0	1	1	0	0	0	0	0
18	0	0	0	0	1	1	0	0	0	1
19	0	0	1	1	0	1	0	0	0	1
20	1	0	0	0	1	1	1	1	1	1
21	1	0	0	0	1	0	0	0	0	0

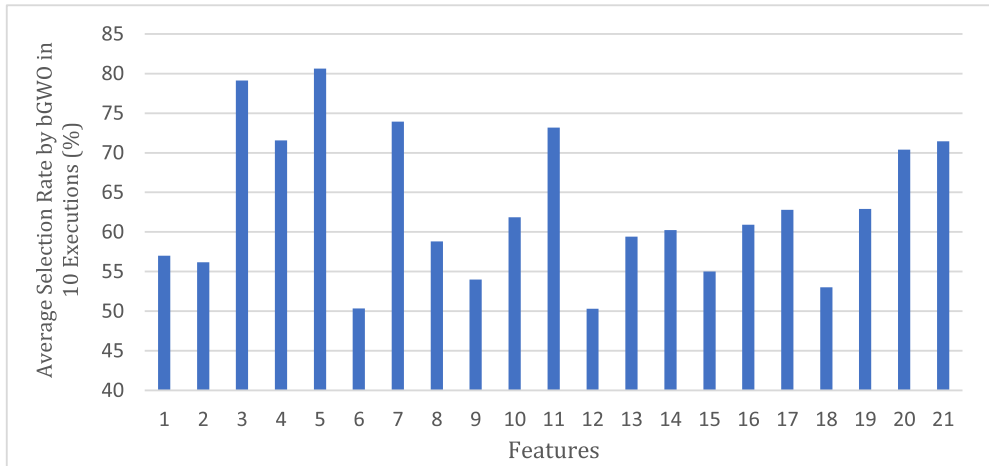


Fig. 16. The selection rate of the 21 features by the bGWO in the KC1 dataset during 10 executions.

techniques, ANN and KNN perform better than the other algorithms. All things considered, the suggested bGWO (as the feature selector) significantly improves the ML algorithm's performance in the software fault prediction issue.

4.5. Discussion of the third experiment's results (JM1 dataset)

Using the JM1 dataset, the third set of experiments has been carried out in response to the RQs. The training and test data sets for the third experiment were taken from the JM1 data set. JM1 dataset includes 21 features and 10,885 records. A train set consists of 7619 recordings, whereas a test dataset includes 3266 records. The performance of the classifiers made using various ML algorithms with and without bGWO is shown in Fig. 18; this Figure displays the performance requirements of the defect predictors (classifiers) developed using the train set with 1619 records by ANN, DT, KNN, NB, and SVM. In the raw train set, every feature (21 features) has been considered. The most accurate and precise classifier is the one produced by KNN. In terms of accuracy and precision, the fault predictor produced by the KNN performs better than the other prediction models. Unnecessary features in the training dataset might lead to overfitting in ML algorithms.

The JM1 dataset has been subjected to the second phase of the third experiment using the bGWO and ML algorithms. The test results of the classifiers created by ML algorithms with bGWO (that was used to determine which features are most effective for the JM1 dataset) are shown in Fig. 18. The accuracy of the classifiers built by KNN and SVM with the usage of bGWO are 93.18 % and 91.20 %, respectively. Indeed, the use of bGWO in the ML algorithms has enhanced the accuracy and precision of the created

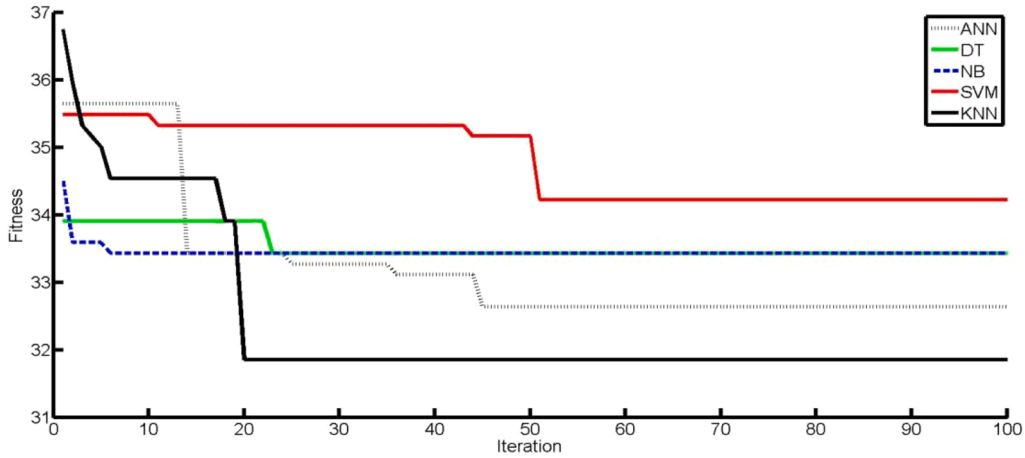


Fig. 17. Convergence of the bGWO for finding effective features with different ML algorithms in the KC1 dataset.

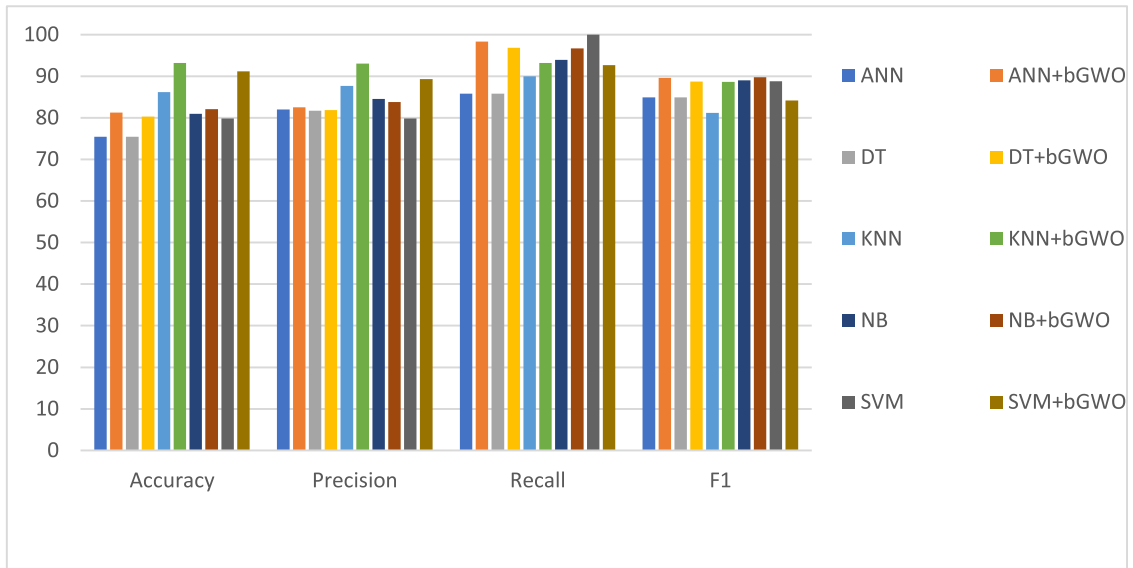


Fig. 18. The effects of the developed bGWO on the performance of the ML algorithms in the JM1 dataset.

classifiers. For another example, the recall and F1 of the fault predictor created by ANN are 85.82 % and 84.89 %, respectively; whereas these figures have been increased to 98.30 % and 89.63 % when bGWO was used as the feature selection. Similar to the other datasets, the experiments demonstrate that the ML algorithms perform better when bGWO is used as a feature selector in the JM1 dataset. It can be concluded that the efficiency of the software defect predictors with the bGWO (as feature selector) is superior to the prior approach by analyzing the results shown in Fig. 18. Indeed, the proposed feature selector makes considerable enhancement to the efficiency of the fault predictors created by different ML algorithms. ML+bGWO achieves superior results since it utilizes the chosen effective qualities. The proposed bGWO method for selecting valuable features works well on the ML algorithms using the JM1 dataset. The fault predictors created by the improved ML (ML+bGWO) have greater accuracy, precision, recall, and F1 than any other model. Overall, the JM1 dataset experiments' outcomes demonstrate that the proposed bGWO for efficient feature selection functions rather well.

The average performance of the fault predictors generated by ML algorithms on the JM1 dataset, with and without feature selection, is shown in Fig. 19. The average accuracy of fault predictor generated by the pure ML and ML+bGWO algorithms are 79.57 % and 85.60 %, respectively. Comparable results were obtained for the other performance criteria. The average precision of the classifiers generated by ML algorithms with and without the recommended feature selector (ML+bGWO) is around 83.14 % and 86.09 %, respectively. As for the results, the classifiers created with the recommended method (ML+bGWO) have an average recall metric value that is noticeably higher than the ML techniques used in isolation. Ultimately, the classifiers generated by the proposed method perform better than those generated by ML algorithms that do not choose features. The proposed bGWO finds and eliminates

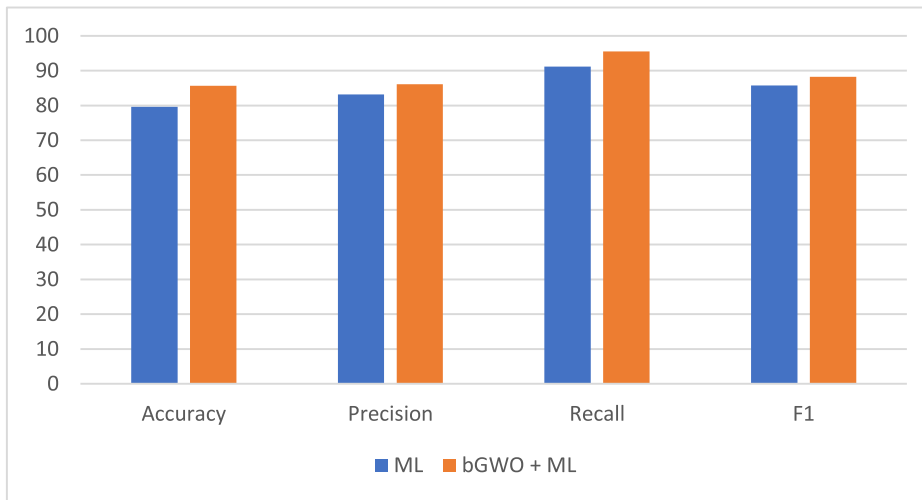


Fig. 19. The average values of the performance criteria of ML (ANN, DT, KNN, and NB) algorithms with and without bGWO as the feature selection in the JM1 dataset.

ineffective features, improving the accuracy of the created fault predictor. Fig. 20 shows how many effective features the recommended bGWO found using different ML techniques. The proposed bGWO selects the most effective features from the 21 features in the JM1 dataset. Table 1 shows the main features of the previous methods.

The recommended approach's average performance was calculated on JM1 after ten times executions, and it showed good detection performance. Fig. 21 displays the average number of features selected by the binary ACO and bGWO throughout the course of 10 runs in the JM1 dataset. When the ANN was used as the ML algorithm, as shown in Fig. 21, bGWO and ACO selected 12 features. In the DT algorithm, bGWO and ACO selected 4 and 9 features, respectively; in the KNN classification algorithm, these values are 7 and 8. In the NB classification process, bGWO and ACO selected 8 and 6 features, respectively. Overall, compared to the ACO, the average number of chosen features by the proposed bGWO is significantly smaller.

Table 9 presents the features of the JM1 dataset that the bGWO selected and refined using several ML algorithms, resulting in the optimal outcomes following ten times iterations. Ten runs have been performed on each ML approach. The features that met the highest assessment criteria were selected as the most effective ones. Fig. 22 shows the bGWO's feature selection rate over 10 executions of the proposed method on the JM1 data set. The following are the ten most useful features that bGWOs in the KC1 dataset have chosen based on the findings:

1. Halstead total Operators + Operands
2. McCabe Design Complexity
3. Halstead Number of Delivered Bugs
4. Branch Count
5. Total Operand
6. McCabe Essential Complexity
7. McCabe Design Complexity
8. Halstead Program Length
9. Number of Unique operators
10. Line of Code and Comment

To show the stable state of the proposed technique, the proposed method was run 10 times on the JM1 dataset. The results show improved metrics for accuracy, precision, recall, and F1. The proposed method considers 21 features and trains the ML algorithm using the selective features by the bGWO. Throughout bGWO's output, the subset of certain features has been selected to create the fault predictor. The chosen features have the most impact on the generated classifier's accuracy and precision. Finally, Fig. 23 shows how the proposed bGWO converges to the optimal features in the JM1 dataset. The performance of the bGWO varies with various ML algorithms. As demonstrated by Fig. 23, the ANN+bGWO and NB+bGWO perform optimally. ANN and NB outperform the other algorithms in terms of fault prediction accuracy. Overall, the performance of the ML algorithm in the software fault prediction problem is much enhanced by the proposed bGWO (as the feature selector).

4.6. Discussion of the third experiment's results (KC2 dataset)

The fourth series of experiments has been conducted in response to the RQs using the KC2 dataset. The KC2 data set was used to create the training and test data sets for the fourth experiment. The KC2 dataset has 522 records and 21 features. A test dataset has 157

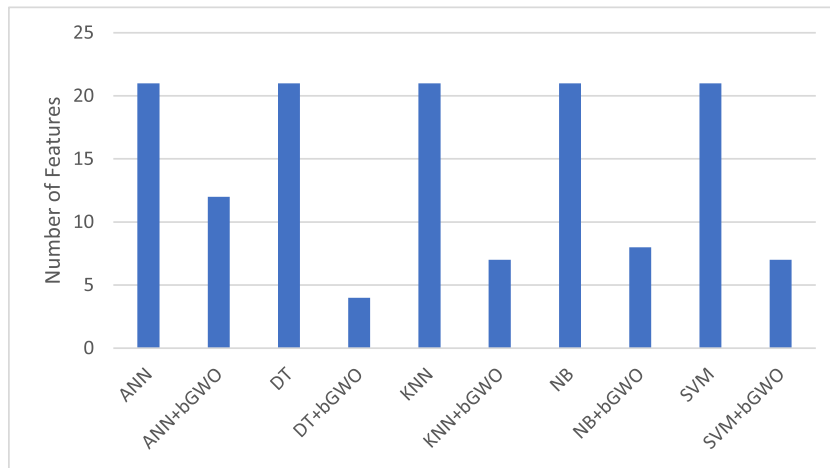


Fig. 20. The average number of selected features by bGWO in different ML algorithms in the JM1 dataset in 10 times executions.

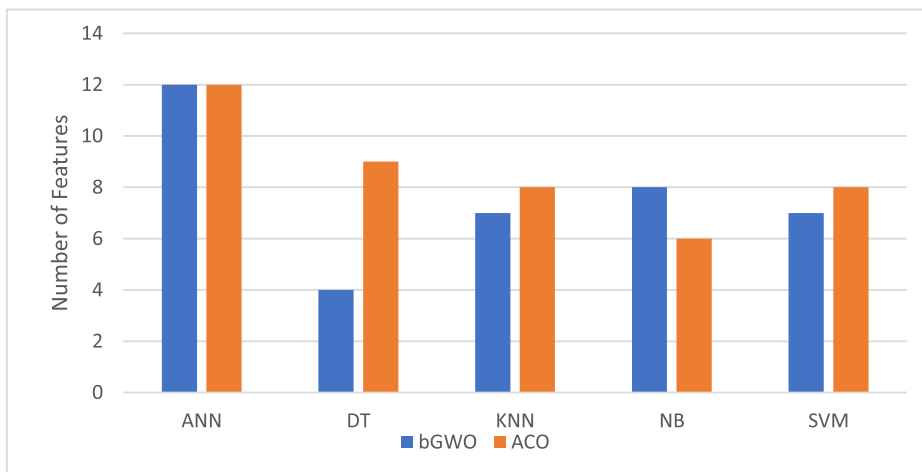


Fig. 21. The average number of the selected features by ACO and bGWO in different ML algorithms in the JM1 dataset.

records, whereas a train set has 365 records. Fig. 24 shows a comparison of ML algorithms' performance metrics with and without feature selectors on the KC2 dataset. The ML+bGWO methods beat the traditional ML algorithms in terms of accuracy, precision, recall, and F1 thanks to the KC2 dataset's chosen features. Fig. 24 presents a comparison of the fault predictors generated by the ML+bGWO and ML techniques. ML+bGWO yields superior results since it utilizes the chosen effective features. Every feature (21 features) in the raw train set has been taken into account. The KNN-produced classifier is the most accurate and precise. Overfitting of ML algorithms might be caused by ineffective features in the KC2 training dataset. Using the bGWO and ML algorithms, the KC2 dataset has been put through the second part of the fourth experiment.

With the use of bGWO, the classifiers constructed using KNN and SVM have accuracy and precision of 93.33 % and 88.53 %, respectively. The use of bGWO in ML algorithms has improved the precision and accuracy of the resulting classifiers (fault predictors). As an additional illustration, the recall and F1 of the fault predictor produced by ANN without bGWO are 90.16 % and 86.27 %, respectively; however, when bGWO was employed as the feature selection, these numbers rose to 98.42 % and 91.91 %. The studies show that, like with the other datasets, in the KC2 dataset, ML algorithms perform better when bGWO is employed as the feature selector. Analyzing the results shown in Fig. 24, it can be inferred that the software defect predictors using the bGWO (as feature selector) are more efficient than the previous method. The suggested feature selection significantly improves upon the effectiveness of the defect predictors produced by various ML algorithms. The proposed bGWO method for selecting optimal features works well on ML algorithms when applied to the KC2 dataset. The fault predictors generated by the enhanced ML (ML+bGWO) have the highest F1, recall, accuracy, and precision when compared to earlier models. Overall, the results of experiments on the KC2 dataset demonstrate that the proposed bGWO for efficient feature selection works rather well.

Fig. 25 displays the average performance of the fault predictors produced by ML methods on the KC2 dataset, both with and without feature selection. The fault predictors produced by the pure ML and ML+bGWO have an average accuracy of 82.76 % and

Table 9

The selected features by bGWO and ACO in different ML algorithms in JM1 in the best result from 10 executions.

#	ANN		DT		KNN		NB		SVM	
	bGWO	ACO	bGWO	ACO	bGWO	ACO	bGWO	ACO	bGWO	ACO
1	0	1	1	1	0	0	0	1	0	0
2	1	0	0	0	0	1	0	1	0	1
3	1	0	1	1	0	1	0	0	0	1
4	1	1	0	1	1	0	0	1	1	0
5	1	0	1	1	1	1	1	1	0	0
6	1	1	0	0	0	1	0	0	0	0
7	1	1	0	0	0	0	1	0	1	1
8	1	0	0	0	0	1	1	0	0	1
9	0	1	1	1	0	0	1	0	0	1
10	1	0	0	1	0	0	1	1	0	0
11	1	1	0	1	1	0	1	0	1	0
12	1	1	0	1	0	1	1	1	0	0
13	0	1	1	0	0	1	1	0	0	1
14	1	0	0	0	0	1	0	1	1	0
15	0	1	0	1	0	1	1	0	0	1
16	1	1	1	1	0	0	1	0	0	0
17	1	1	1	0	0	0	0	1	0	1
18	1	0	0	1	1	1	0	0	0	1
19	1	1	0	1	0	1	0	0	0	1
20	1	0	0	0	0	0	1	0	1	1
21	1	0	0	0	1	1	1	0	1	1

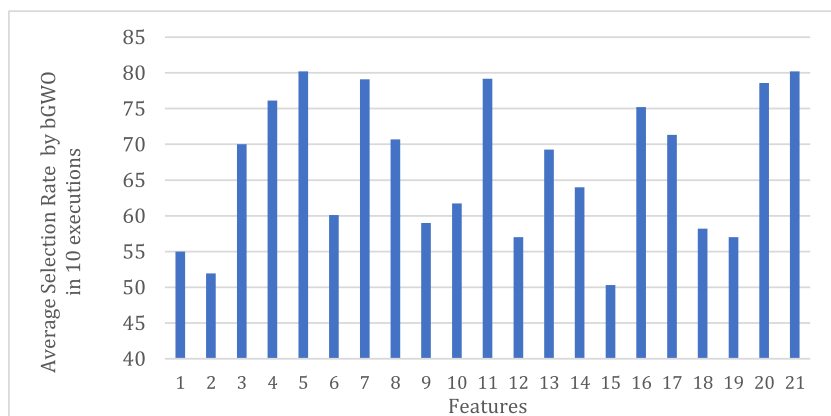


Fig. 22. The selection rate of the 21 features by the bGWO in the JM1 dataset during ten times executions.

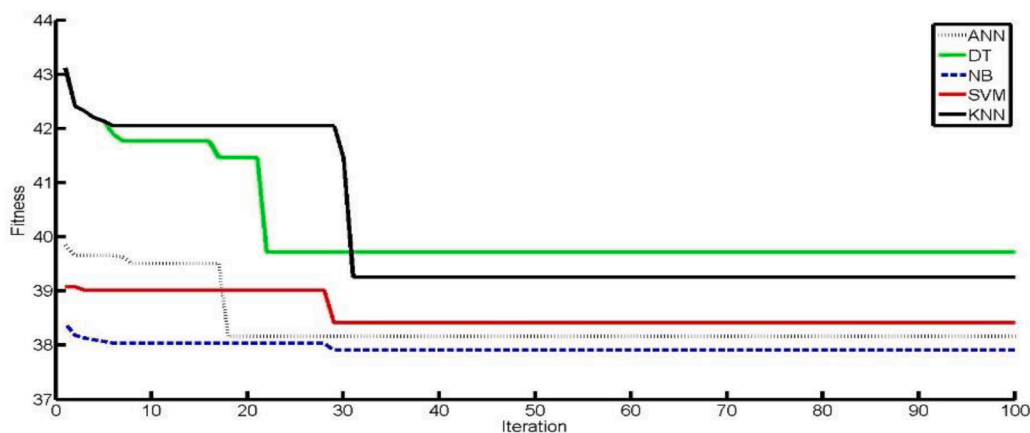


Fig. 23. Convergence of the bGWO for finding effective features with different ML algorithms in the JM1 dataset.

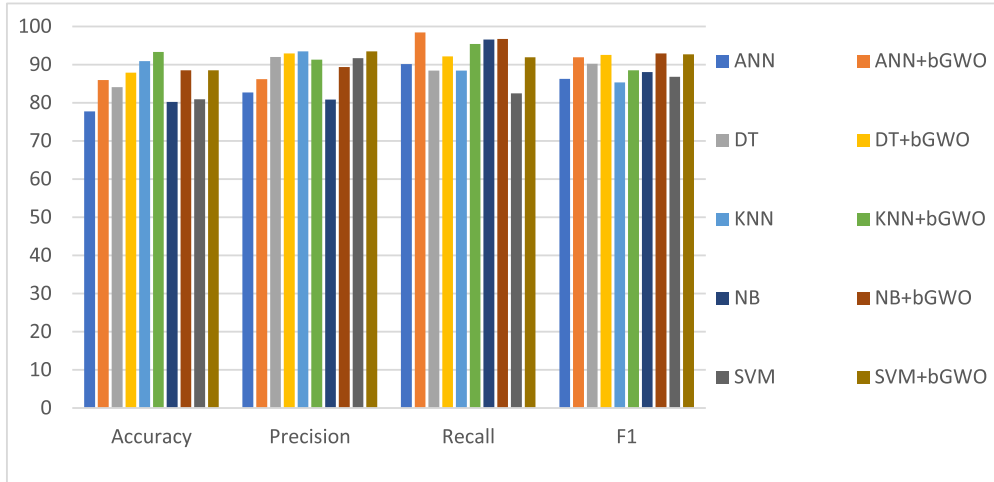


Fig. 24. The effects of the developed bGWO on the performance of the ML algorithms in the KC2 dataset.

88.85 %, respectively. For the remaining performance criteria, comparable outcomes were attained. The classifiers produced by ML techniques, both without and with the suggested feature selection (ML+bGWO), had an average precision of around 88.14 % and 90.65 %, respectively. In terms of outcomes, the classifiers developed using the suggested approach (ML+bGWO) have an average recall metric value that is notably greater than the pure ML methods used separately. In the end, the fault predictor produced by the suggested technique (ML+bGWO) outperforms those produced by ML algorithms without feature selection. By identifying and removing ineffective features, the suggested bGWO raises the fault predictor's accuracy. The number of useful features that the suggested bGWO discovered using various ML methods is displayed in Fig. 26. Out of the 21 features in the KC2 dataset, the most useful features are chosen by the suggested bGWO.

After ten executions, the average performance of the suggested method was determined on the KC2 dataset. The average number of features chosen during a period of 10 runs in the KC2 dataset by the binary ACO and bGWO is shown in Fig. 27. As seen in Fig. 27, when the ANN was utilized as the ML method, bGWO and ACO chose 8 and 7 features respectively. The proposed bGWO and ACO choose 12 and 9 features in the DT ML method, respectively; these numbers are 9 and 9 in the KNN classification algorithm. 7 and 8 features, respectively, were chosen by bGWO and ACO throughout the NB algorithm. Overall, the average number of features selected by the proposed bGWO is substantially fewer than that of the ACO.

Table 10 lists the KC2 dataset's features that the bGWO chose and improved using a variety of ML techniques, arriving at the best results after ten iterations. For every machine learning strategy, ten runs have been made. The aspects that were determined to be the

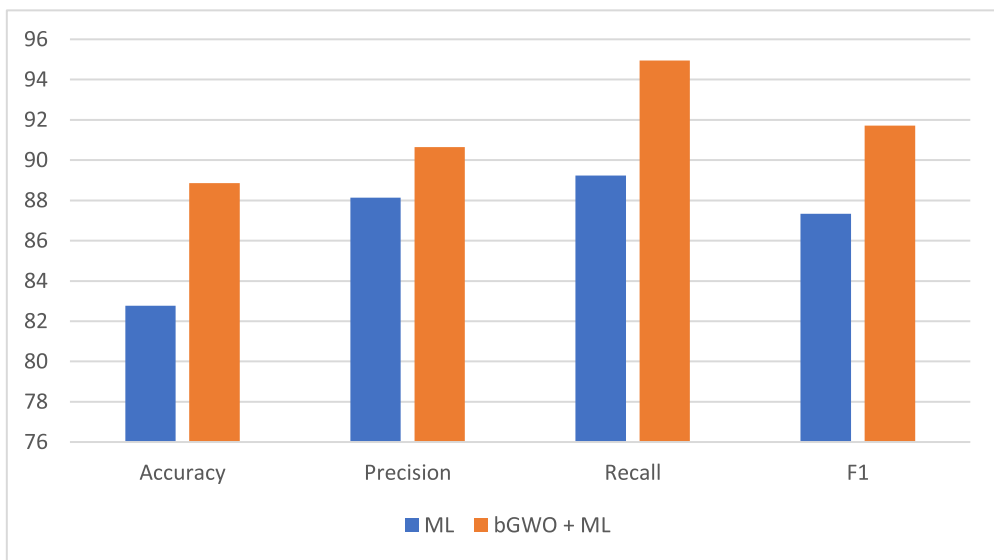


Fig. 25. The average values of the performance criteria of ML (ANN, DT, KNN, and NB) algorithms with and without bGWO as the feature selector in the KC2 dataset in 10 times executions.

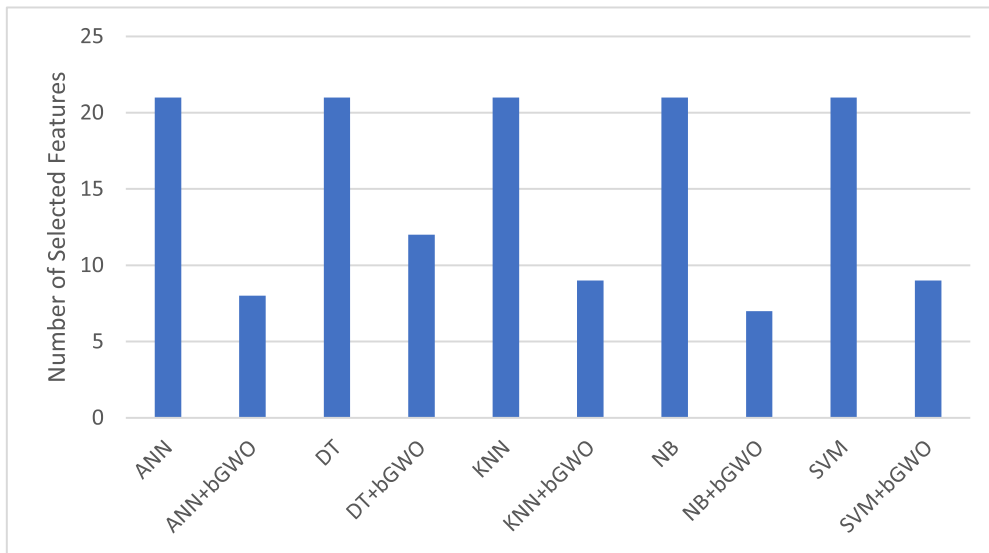


Fig. 26. The average number of selected features by bGWO in different ML algorithms in the KC2 dataset in 10 times executions.

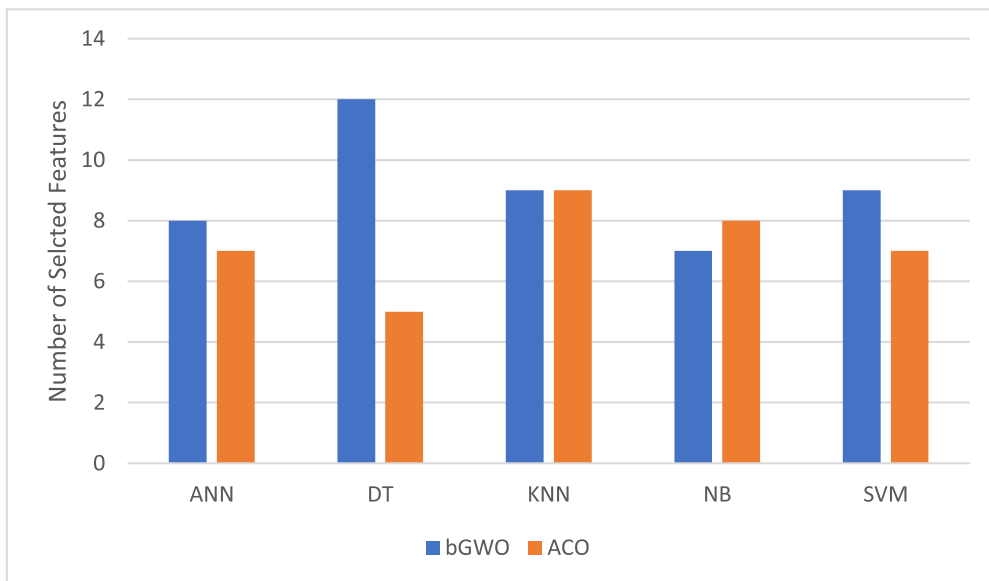


Fig. 27. The average number of selected features by ACO and bGWO in different ML algorithms in the KC2 dataset in ten times executions.

most beneficial were those that satisfied the highest evaluation standards. The feature selection rate of the bGWO is displayed in Fig. 28 after ten iterations of the suggested approach on the KC2 data set. Based on the results, bGWOs in the KC2 dataset selected the top 10 features that are most valuable, which are as follows:

1. Halstead's "difficulty"
2. Number of total operators
3. Branch Count
4. Number of unique operators
5. McCabe essential complexity
6. unique operands
7. Halstead's count of blank lines
8. Effort to write the program
9. Program length"

Table 10

The selected features by bGWO and ACO in different ML algorithms in KC2 in the best result from 10 executions.

#	ANN		DT		KNN		NB		SVM	
	bGWO	ACO	bGWO	ACO	bGWO	ACO	bGWO	ACO	bGWO	ACO
1	0	1	0	0	0	1	1	0	0	0
2	1	1	0	1	0	1	0	1	0	0
3	0	1	1	1	1	0	1	1	1	1
4	0	0	1	0	0	0	1	1	1	0
5	0	0	0	0	1	1	1	0	0	0
6	0	1	1	0	1	1	0	0	0	0
7	1	1	1	1	0	1	0	1	1	0
8	1	0	1	1	1	1	0	1	0	1
9	1	0	0	1	0	1	0	0	1	0
10	1	1	1	0	0	0	1	1	0	0
11	0	1	1	0	1	0	0	0	0	0
12	1	1	0	1	1	1	0	1	0	1
13	1	0	1	1	0	0	0	0	0	0
14	0	0	1	0	0	0	1	1	0	1
15	1	1	1	0	0	0	0	0	1	1
16	1	0	0	1	1	1	0	0	0	0
17	0	1	1	1	0	0	1	1	1	1
18	1	1	1	0	0	0	0	0	1	0
19	1	0	1	0	0	0	1	0	1	0
20	1	1	0	1	0	1	0	1	0	1
21	1	0	0	0	1	1	1	1	0	0

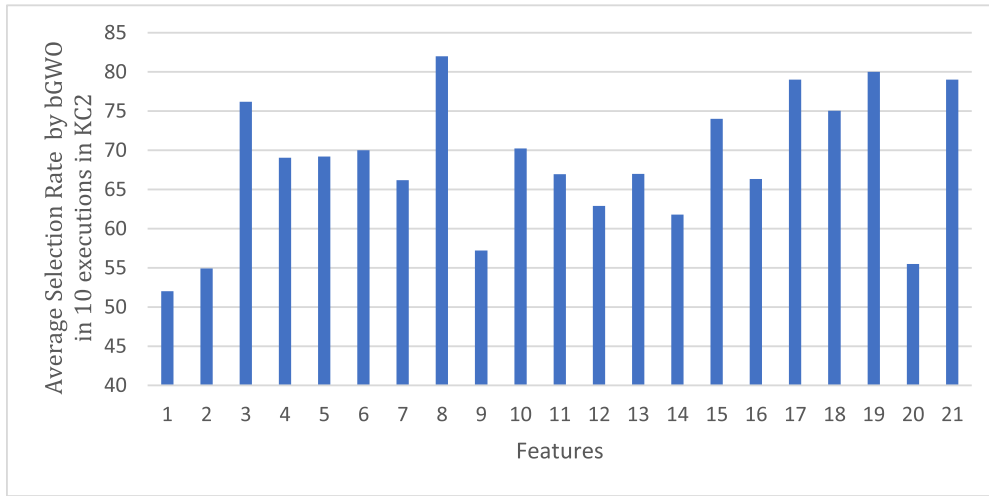


Fig. 28. The selection rate of the 21 features by the bGWO in the KC2 dataset during ten times executions.

10. Total operators + operands

The suggested strategy was applied ten times on the KC2 dataset to demonstrate its stable status. The outcomes demonstrate enhanced accuracy, precision, recall, and F1 measures. The suggested approach takes into account 21 features and uses bGWO's selected features to train the ML algorithm. A selection of certain features from bGWO's output has been chosen to build the fault predictor. The accuracy and precision of the resulting fault predictor are mostly determined by the selected features. The relative contribution of each feature to the fault predictor in the KC2 dataset is displayed in Fig. 28. Lastly, Fig. 29 illustrates the convergence of the suggested bGWO to the KC2 dataset's optimal features. bGWO's performance varies with different ML techniques. Fig. 29 shows that the ANN+bGWO and SVM+bGWO operate at their best in terms of convergence speed. When compared to the other algorithms, ANN, SVM, and NB perform better in terms of accurate fault prediction. Overall, the suggested bGWO (as the feature selector) significantly improves the performance of the ML algorithm in the software prediction issue.

4.7. Discussion of the third experiment's results (PC1 dataset)

Using the PC1 dataset, the final series of experiments have been carried out in response to the RQs. The training and test data sets

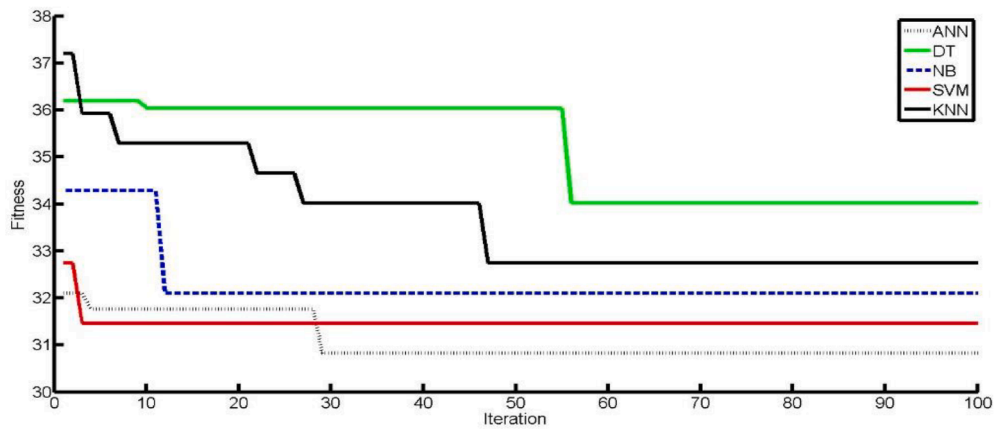


Fig. 29. Convergence of the bGWO for finding effective features with different ML algorithms in the KC2 dataset.

for the final experiment were made using the PC1 data set. There are 21 features and 1109 entries in the PC1 dataset. There are 333 entries in the test dataset and 776 in the train set. Fig. 30 compares the performance metrics of ML algorithms with and without feature selectors. Because of the features selected from the PC1 dataset, the ML+bGWO approaches outperform the conventional ML algorithms in terms of accuracy, precision, recall, and F1.

The second phase of the fifth (final) experiment has been conducted on the PC1 dataset using the bGWO and ML algorithms. The fault predictor created by ML algorithms using bGWO is tested, and the results are shown in Fig. 30. The classifiers built using KNN and ANN have 97.81 % and 95.79 % accuracy and precision, respectively, when bGWO is used. An additional example would be the fault predictor generated by pure ANN, which has a recall and F1 of 99.66.00 % and 95.54 %, respectively; however, these percentages increased to 100.00 % and 97.84 % when bGWO was used as the feature selector. According to the research, bGWO is a better feature selector for ML algorithms when used in the PC1 dataset, just like it is in the other datasets. It is possible to deduce from Figs. 30 and 31 that the software fault predictors that use the bGWO are more effective than the earlier approach. Because ML+bGWO makes use of the selected effective features, it produces better outcomes. When used on ML algorithms, the suggested bGWO approach for choosing the best features performs admirably on the PC1 dataset. When compared to previous models, the fault predictors produced by the improved ML (ML+bGWO) had the best F1, recall, accuracy, and precision. By identifying and removing ineffective features, the suggested bGWO raises the fault predictor's performance. The number of useful features that the suggested bGWO discovered using various ML methods in the PC1 dataset is displayed in Fig. 32. Out of the 21 features in the PC1 dataset, the most useful features are chosen by the suggested bGWO.

The average performance of the fault predictors generated by ML techniques on the PC1 dataset is shown in Fig. 25, with and without feature selection. The average accuracy of the fault predictors generated by the ML+bGWO and pure ML is 95.40 % and 88.82 %, respectively. Similar results were obtained for the other performance criteria. Results indicate that the fault predictor created with the recommended technique has an average recall metric value that is much higher than the results of utilizing the pure ML approaches alone. Ultimately, the fault predictor generated by the proposed method (ML+bGWO) performs better than those generated by ML methods without feature selection.

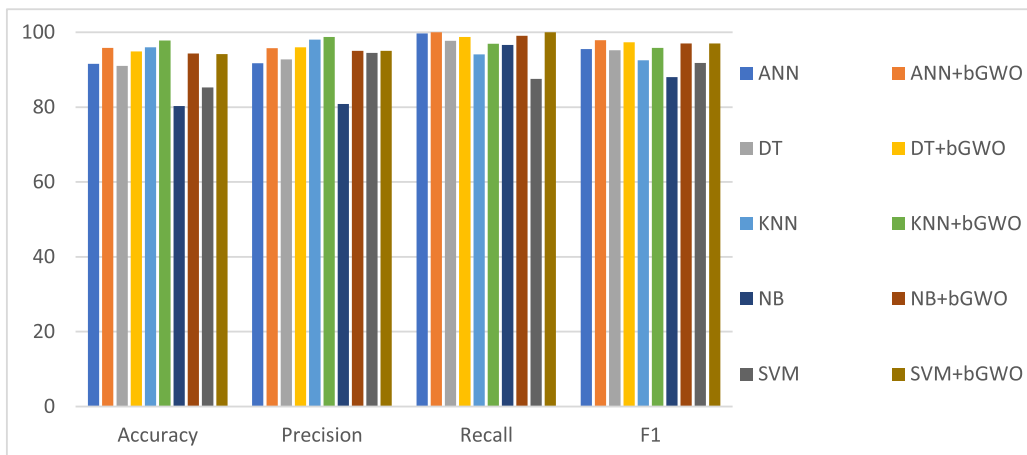


Fig. 30. The effects of the developed bGWO on the performance of the ML algorithms in the PC1 dataset.

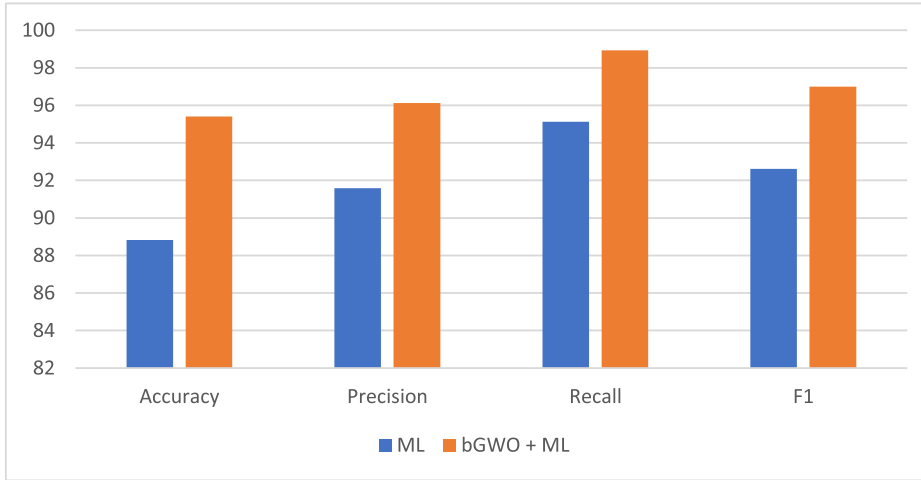


Fig. 31. The average values of the performance criteria of ML (ANN, DT, KNN, and NB) algorithms with and without bGWO as the feature selection in the PC1 dataset.

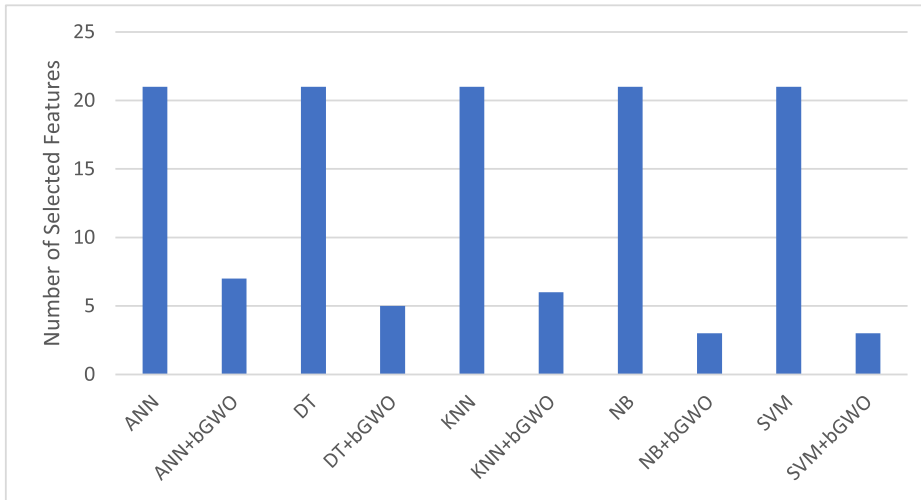


Fig. 32. The number of selected features by bGWO in different ML algorithms in the PC1 dataset.

The recommended method's average performance on the PC1 dataset has been determined after 10 executions. Fig. 33 displays the average number of features selected during a period of 10 runs in the PC1 dataset by the binary ACO and bGWO. As seen in Fig. 33, bGWO and ACO selected 7 and 9 features, respectively, when the ANN was used as the ML technique. In the DT+bGWO technique, 5 features were selected, and 11 features were selected by DT+ACO; these values are 6 and 10 in the KNN classification algorithm. Throughout the NB+bGWO three features were selected as optimal features; while, throughout NB+ACO eight features were selected. Overall, compared to the intended bGWO, the average number of features chosen is significantly lower than ACO.

The PC1 dataset's features that the bGWO selected and refined via 10 rounds utilizing different ML algorithms are listed in Table 11. Ten executions have been performed for each ML technique. The features that met the strictest evaluation criteria were found to be the most advantageous. After ten executions of the proposed method on the PC1 data set, Fig. 34 shows the feature selection rate of the bGWO in PC1. The top 10 most useful features in the PC1 dataset were determined by bGWOs based on the findings, and they are as follows:

1. McCabe "design complexity"
2. Halstead "difficulty"
3. Branch Count
4. Halstead "Number of Delivered Bugs"
5. Halstead's time estimator: time to write a program
6. Total Number of operators + operands

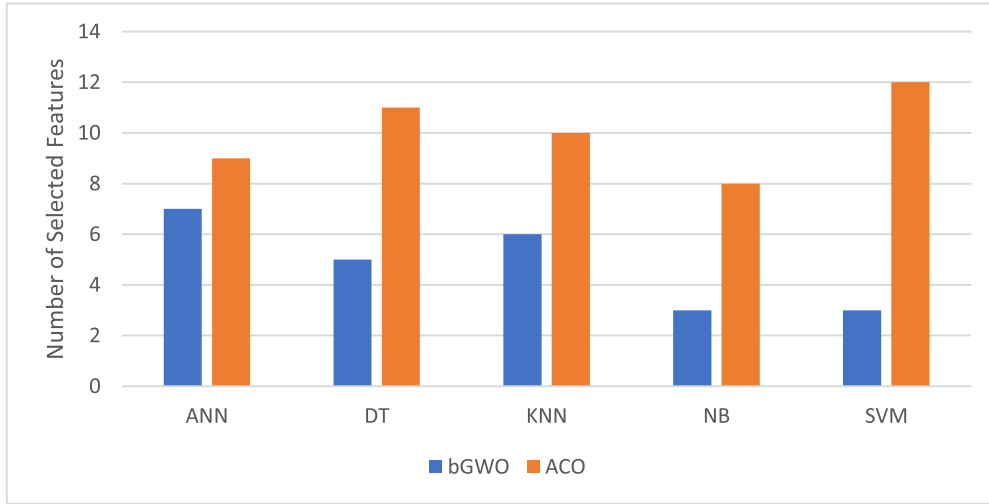


Fig. 33. The number of selected features by ACO and bGWO in different ML algorithms in the PC1 dataset.

Table 11

The selected features by bGWO and ACO in different ML algorithms in PC1 in the best result from 10 executions.

#	ANN		DT		KNN		NB		SVM	
	bGWO	ACO	bGWO	ACO	bGWO	ACO	bGWO	ACO	bGWO	ACO
1	1	1	0	1	0	1	1	0	0	1
2	0	0	0	0	0	0	1	1	0	0
3	0	1	1	0	1	0	0	1	0	1
4	1	1	1	1	1	1	0	1	1	1
5	1	0	1	0	0	1	1	1	0	1
6	0	1	1	1	1	1	0	0	0	1
7	0	0	0	1	1	0	0	1	1	0
8	1	0	0	0	1	1	1	1	1	0
9	1	1	0	0	0	0	1	0	0	1
10	0	0	0	0	0	1	0	1	1	0
11	1	0	1	0	0	0	1	0	0	1
12	1	1	0	1	0	0	1	1	0	0
13	0	1	1	1	1	0	0	0	0	1
14	1	0	1	1	0	1	0	0	0	1
15	0	1	0	1	1	1	0	1	1	1
16	0	1	0	0	0	0	1	0	1	0
17	1	0	1	1	0	0	0	1	0	1
18	0	1	1	1	0	1	1	1	0	0
19	1	0	0	1	0	0	0	0	0	1
20	1	0	1	0	0	1	0	0	0	1
21	1	1	0	0	1	1	0	0	1	1

7. Halstead's line count
8. Total Number of Operand
9. Number of unique operands
10. Number of unique operators

To show that the proposed technique is stable, it was applied 10 times on the PC1 dataset. The results show improved F1, recall, accuracy, and precision. The proposed method selects features by bGWO and utilizes them to train the machine learning algorithm while accounting for 21 features. The fault predictor has been constructed using a subset of certain features selected from the output of bGWO. The chosen features mostly dictate the accuracy and precision of the fault predictor that is produced. Finally, Fig. 35 shows how the proposed bGWO converges to the best features of the PC1 dataset. The performance of bGWO changes depending on the ML method used. The optimal convergence speed of the SVM+bGWO and KNN+bGWO is higher than the other methods. In terms of precise fault prediction, ANN, and KNN outperform the other methods. All things considered, the proposed bGWO (as the feature selector) greatly enhances the ML algorithm's performance in the software prediction problem.

The accuracy of the fault predictors produced by several ML algorithms using various datasets is displayed in Fig. 36. Compared to the other fault predictors produced by the other approaches, the one generated by the suggested method in the PC1 dataset has a

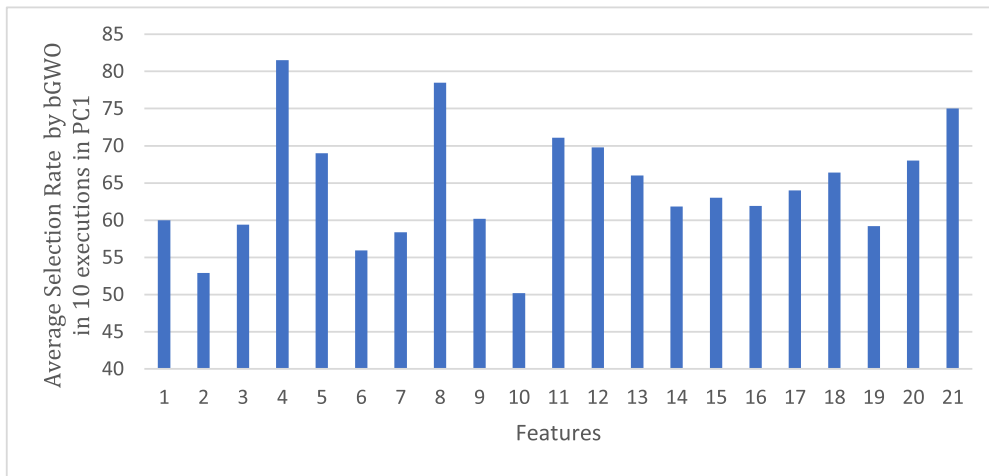


Fig. 34. The selection rate of the 21 features by the bGWO in the PC1 dataset during ten times executions.

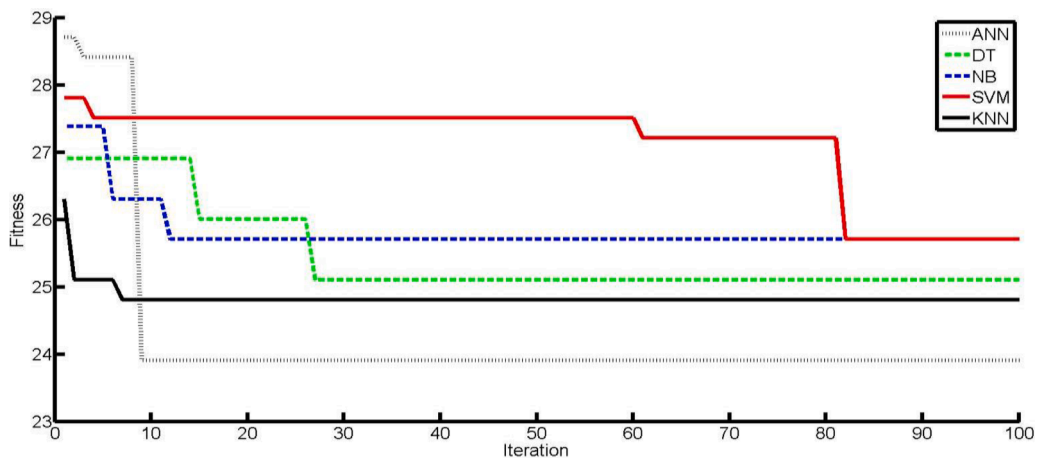


Fig. 35. Convergence of the bGWO for finding effective features with different ML algorithms in the PC1 dataset.

greater accuracy of 95 %. The fault predictor produced by the suggested method yields an accuracy of almost 89 % in the KC2 dataset; in the JM1 and CM1 datasets, this value is 85 % and 92 %, respectively. In general, the accuracy and performance of the suggested approach outperforms those of the other methods. The developed bGWOA is what led to this improvement. The training dataset's most useful features are identified by the created bGWOA. The software built performs better and is more accurate when the ineffective attributes are removed.

5. Conclusion

Creating a software fault predictor has a great impact on the software development activity and also on its quality. In this paper, a binary version of the gray wolf optimization algorithm was developed to select the most effective features of the fault prediction dataset. An effective and efficient fault predictor was created using the filtered training set with optimal features. The efficiency of the created fault predictor was improved by identifying the smallest subset and the most effective features. The optimal features of the standard fault prediction datasets were identified by the proposed bGWO. The results of experiments on the NASA data sets indicate that the proposed bGWO is an effective feature selector for creating ML-based fault predictors. The most effective features selected by the bGWO are McCabe design complexity, total number of operators and operands in the code, Halstead program length, Halstead difficulty, number of branches in the source code, line of code and comments, and number of unique operators and operands. In this research, the created fault predictor by combining the bGWO and ML algorithms provides 95.40 % accuracy, 96.11 % precision, 98.93 % recall, and 97.02 % F1 on average. Using the proposed method on other components and service-based software datasets is one of the future studies.

Approximately, all of 21 features in the NASA fault-prediction datasets are related to the source code of program; evaluating the effects of the design level features in the performance of the created fault predictors can be considered as one of future studies. All of 21

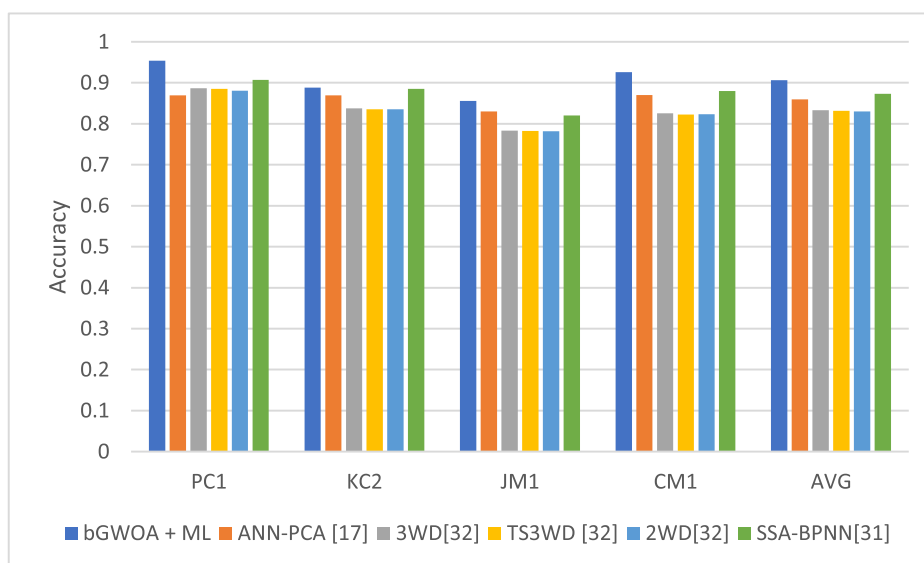


Fig. 36. Comparison of different fault predictors' accuracy.

features in the NASA fault-prediction datasets are related to complexity metrics of the functional and object-oriented structures. The features of the novel programming structures (such as API and service-based programming) can be taken into consideration in the fault prediction datasets. The first population of the proposed method is randomly generated; chaos and heuristic based population initialization may improve the performance of the proposed feature selector. Furthermore, the removal of a feature by the bGWO method does not mean that the feature does not affect the prediction; but it indicates that more effective features empower the learning algorithms. Furthermore, combining another novel heuristic algorithm with the ML algorithms to create a software fault predictor can be considered as another future study. Using the proposed method to create classifiers on other datasets in medical or non-medical applications is another future research. Moreover, the ML-based fault predictors can benefit from the integration of reinforcement learning methods and chaotic maps to improve performance. The recently proposed heuristic and ML algorithms such as [23,24] can be used as feature selector in software fault prediction methods. Improving the accuracy of the software predictors using deep learning techniques can be considered as another future study.

Ethical approval

All authors state that this study is the authors' own original work, which has not been previously published elsewhere. The paper is not currently being considered for publication elsewhere. The paper reflects the authors' own research and analysis in a truthful and complete manner.

Funding

The authors declare that no funds were received during the preparation of this manuscript.

Availability of data and materials

The datasets generated during study and the implemented code are freely available by the following link: https://drive.google.com/drive/folders/18TSA-wfgUkWtI6LX9GuJvU8_oIKyDPv8?usp=drive_link.

Author agreement statement

We the undersigned declare that this manuscript is original, has not been published before and is not currently being considered for publication elsewhere. We confirm that the manuscript has been read and approved by all named authors and that there are no other persons who satisfied the criteria for authorship but are not listed. We further confirm that the order of authors listed in the manuscript has been approved by all of us. We understand that the Corresponding Author is the sole contact for the Editorial process. He is responsible for communicating with the other authors about progress, submissions of revisions and final approval of proofs Signed by all authors as follows:

Declaration of competing interest

We are pleased to submit an original research article entitled “A Software Defect Prediction Method Using Binary Gray Wolf Optimizer and Machine Learning Algorithms”, by Hao Wang, Bahman Arasteh, Keyvan Arasteh, Farhad Soleimanian Ghar-ehchopogh, and Alireza Rouhi for consideration for publication in the International Journal of Computers and Electrical Engineering. We (authors) confirm that we have not any conflict of interest regarding the submission of this manuscript to the journal. I should state that the authors are preparing articles in their personal capacity.

Data availability

Data will be made available on request.

References

- [1] Arasteh B. Software fault-prediction using combination of neural network and naive bayes algorithm. *J Netw Technol* 2018;9(3):94–101. <https://doi.org/10.6025/jnt/2018/9/3/94-101>.
- [2] Al-Laham M, Kassaymeh S, Al-Betar MA, Makhadmeh SN, Albashish D, Alweshah M. An efficient convergence-boosted salp swarm optimizer-based artificial neural network for the development of software fault prediction models. *Comput Electric Eng* 2023;111:108923. <https://doi.org/10.1016/j.compeleceng.2023.108923>. Part A ISSN 0045-7906.
- [3] Song Q, Jia Z, Shepperd M, Ying S, Liu J. A general software defect-proneness prediction framework. *IEEE Trans Softw Eng* 2011;37(3):356–70.
- [4] Papa PJ, Rosa GH, André N, Luis Afonso CS. Feature selection through binary brain storm optimization. *Comput Electric Eng* 2018;72:468–81. <https://doi.org/10.1016/j.compeleceng.2018.10.013>. ISSN 0045-7906.
- [5] Ghaemi A, Arasteh B. SFLA-based heuristic method to generate software structural test data. *J Softw Evol Proc* 2020;32:e2228. <https://doi.org/10.1002/smr.2228>.
- [6] Shomali N, Arasteh B. Mutation reduction in software mutation testing using firefly optimization algorithm. *Data technologies and applications*, 54; 2020. p. 461–80. <https://doi.org/10.1108/DTA-08-2019-0140>.
- [7] Hosseini MJ, Arasteh B, Isazadeh A, Mohsenzadeh M, Mirzarezaee M. An error-propagation aware method to reduce the software mutation cost using genetic algorithm. *Data technologies and applications*, 55; 2021. p. 118–48. <https://doi.org/10.1108/DTA-03-2020-0073>.
- [8] B. Arasteh B, Najafi J. Programming guidelines for improving software resiliency against soft-errors without performance overhead *Computing* 2018;100: 971–1003. <https://doi.org/10.1007/s00607-018-0592-y>.
- [9] B. Arasteh B, Miremadi SG, Rahmani AM. Developing inherently resilient software against soft-errors based on algorithm level inherent features *J Electron Test* 2014;30:193–212. <https://doi.org/10.1007/s10836-014-5438-8>.
- [10] Iqra Batool B, Tamim Ahmed Khan AK. Software fault prediction using data mining, machine learning and deep learning techniques: a systematic literature review. *Comput Electric Eng* 2022;100:107886. <https://doi.org/10.1016/j.compeleceng.2022.107886>. ISSN 0045-7906.
- [11] Jiang Y, Li M, Zhou Z, Member S. Software defect detection with rous. *J Comput Sci Technol* 2011;26(2):328–42.
- [12] Shuo Wang S, Xin Yao X. Using class imbalance learning for software defect prediction. *IEEE Trans Reliab* 2013;62(2):434–43.
- [13] Galar M, Fern A, Barrenechea E, Bustince H. A review on ensembles for the class imbalance problem: bagging-, Boosting-, and Hybrid-Based approaches mikel. *IEEE Trans Syst MAN, Cybern* 2011;42(4):1–22.
- [14] Anbu M, Anandha GS. Feature selection using firefly algorithm in software defect prediction. *Cluster Comput* 2019;22:10925–34. <https://doi.org/10.1007/s10586-017-1235-3>.
- [15] Mafarja M, Thaher T, Al-Betar MA, et al. Classification framework for faulty-software using enhanced exploratory whale optimizer-based feature selection scheme and random forest ensemble learning. *Appl Intell* 2023;53:18715–57. <https://doi.org/10.1007/s10489-022-04427-x>.
- [16] Yousef AH. Extracting software static defect models using data mining. *Ain Shams Eng J* 2014;6(1):133–44.
- [17] Jayanthi R, Florence L. Software defect prediction techniques using metrics based on neural network classifier. *Cluster Comput* 2019;22(Suppl 1):77–88. <https://doi.org/10.1007/s10586-018-1730-1>.
- [18] Laradji IH, Alshayeb M, Ghouti L. Software defect prediction using ensemble learning on selected features. *Inf Softw Technol* 2015;58:388–402.
- [19] Yucalar F, Ozcift A, Borandag E, Kilinc D. Multiple-classifiers in software quality engineering: combining predictors to improve software fault prediction ability. *Eng Sci Technol Int J* 2020;23(4):938–50. <https://doi.org/10.1016/j.jestech.2019.10.005>. Num.ISSN 2215-0986.
- [20] Promise software engineering repository, [Online Available: <http://promise.site.uottawa.ca/SERepository/datasets-page.html>].
- [21] Mirjalili S, Mirjalili SM, Lewis A. Grey wolf optimizer. *Adv Eng Softw* 2014;69:46–61. <https://doi.org/10.1016/j.advengsoft.2013.12.007>. ISSN 0965-9978.
- [22] Kanan HR, Faez K, Taheri SM. Feature selection using ant colony optimization (ACO): a new method and comparative study in the application of face recognition system. In: *Adv. Data Mining. Theor. Asp. Appl. 7th Ind. Conf. ICDM 2007*. 4597; 2007. p. 63–76.
- [23] Arasteh B, Sadegi R, Arasteh K. Bölen: software module clustering method using the combination of shuffled frog leaping and genetic algorithm. *Data technologies and applications*, 55; 2021. p. 251–79. <https://doi.org/10.1108/DTA-08-2019-0138>.
- [24] Gharehchopogh FS, Abdollahzadeh B, Arasteh B. An improved farmland fertility algorithm with hyper-heuristic approach for solving travelling salesman problem. *CMES-Comput Model Eng Sci* 2023;135(3):1981–2006. <https://doi.org/10.32604/cmesc.2023.024172>.
- [25] Campos J, Costa E, Vieira M. Improving failure prediction by Ensembling the decisions of machine learning models: a case study. *IEEE Access* 2019;7: 177661–74.
- [26] Bal P, Kumar S. Extreme learning machine based linear homogeneous ensemble for software fault prediction. *ICSOF 2018*:103–12.
- [27] He H, Zhang X, Wang Q, Ren J, Liu J, Zhao X, Cheng Y. Ensemble multiboot based on ripper classifier for prediction of imbalanced software defect data. *IEEE Access* 2019;7:110333–43.
- [28] Coelho RA, Dos R, et al. Applying swarm ensemble clustering technique for fault prediction using software metrics. In: *2014 13th International conference on machine learning and applications*, IEEE; 2014. p. 356–61.
- [29] Ryu D, Baik J. Effective harmony search-based optimization of cost-sensitive boosting for improving the performance of cross-project defect prediction. *KIPS Trans Softw Data Eng* 2018;7(3):77–90.
- [30] Wang H, Khoshgoftaar T, Napolitano A. A comparative study of ensemble feature selection techniques for software defect prediction. In: *Ninth international conference on machine learning and applications*. IEEE; 2010. p. 135–40.
- [31] Sofian K, Salwani A, Mohammed A, Mohammed A. Salp swarm optimizer for modeling the software fault prediction problem. *J King Saud Univ - Comput Inf Sci* 2022;34(6):3365–78. <https://doi.org/10.1016/j.jksuci.2021.01.015>. Part B ISSN 1319-1578.
- [32] Li W, Huang Z, Li Q. Three-way decisions-based software defect prediction. *Knowl Based Syst* 2016;91:263–74. <https://doi.org/10.1016/j.knsys.2015.09.035>. ISSN 0950-7051.
- [33] Shao X, Wang H. Nonlinear tracking differentiator based on improved sigmoid function. *Control Theory Appl* 2014;31. 1116e1122.